

Investigation of Similar Patterns prediction with KNN Classifier in Hierarchical Temporal Memory

Master thesis

by

Jayashree Regoti

submitted to obtain the degree of
Master of Engineering (M.Eng.)

at

Frankfurt University Of Applied Sciences
Computer Science and Engineering

Course of Studies
Information Technology

First examiner: Prof. Dr. Pech
Frankfurt University of Applied Sciences

Second examiner: Prof. Dr. A. Lehmann
Frankfurt University of Applied Sciences

Supervisor: Prof. Dr. Damir Dobric
Frankfurt University of Applied Sciences

Frankfurt, November 2024

Contact details:

Jayashree Regoti

Frankfurt University of Applied Sciences

Computer Science and Engineering

jayashree.regoti@stud.fra-uas.de

Prof. Dr. Damir Dobric

Frankfurt University of Applied Sciences

damir.dobric@daenet.de

Prof. Dr. Pech

Frankfurt University of Applied Sciences

Computer Science and Engineering

pech@fb2.fra-uas.de

Prof. Dr. A. Lehmann

Frankfurt University of Applied Sciences

Head of Information Technology (IT)

Master's program

lehmann@e-technik.org

Abstract

In this study, we integrate a K-Nearest-Neighbor (KNN) classifier with Hierarchical Temporal Memory (HTM) through the Neocortex API to analyze pattern similarity in images. The KNN model is trained using pre-labeled input data, which is converted into input images—structured as 30x30 pixel arrays of binary values. This conversion allows for visual and computational representation of input patterns. During the experiment, input images are generated from a 2D array format, facilitating precise control over dimensions and input consistency. For each input image, an output SDR is generated using a spatial pooler, capturing active columns that reflect similarities among patterns.

The methodology involves creating SDR images from both training and testing datasets, encoding these via the spatial pooler, and analyzing their similarity through the KNN classifier. The model's configuration is fine-tuned using specific parameters, such as potential radius and activation threshold, within the HTMConfig. The KNN classifier then learns from the labeled training SDRs and predicts similarities in test SDRs, providing a ranked list of the top three similar matches for each test pattern.

Results demonstrate the system's effectiveness in identifying similar patterns. This approach leverages KNN's simplicity for classification and HTM's robustness in temporal anomaly detection, effectively predicting pattern similarity across repetitive datasets. The system's ability to distinguish similar input patterns supports the potential of combined KNN-HTM architectures in recognizing and categorizing structured data patterns. Future work may focus on expanding input diversity and further optimizing the KNN-HTM integration to enhance predictive precision.

Acknowledgement

I would like to express my sincere gratitude to my academic supervisor, Prof. Dr. Damir Dobric, for providing me with the opportunity to conduct this research and for his valuable guidance throughout the process.

I am also thankful to my supervisors, Prof. Dr. Andreas Pech and Prof. Dr. A. Lehmann, for agreeing to review my thesis project.

Special thanks to Ms. Mahdiah Pirmoradian for being available constantly whenever her support is needed.

Finally, I would like to express my appreciation to my family and friends for their unwavering encouragement and support during this trying time. Their confidence in my skills gave me the willpower to keep going.

To conclude, without the combined efforts and guidance of everyone listed and others who made various contributions, this thesis would not have been able to be finished. I want to express my gratitude to everyone.

Contents

Abstract.....	3
Acknowledgement	4
Contents	5
List of Figures	7
List of Abbreviations	8
1. Introduction.....	9
1.1 Hierarchical Temporal Memory (HTM)	9
1.2 Spatial Pooler.....	10
1.3 Temporal memory	10
1.4 How KNN Works	11
1.5 Sparse Distributed Representation	11
1.6 Required Technologies	12
2. Literature Review.....	14
2.1 The Neocortex and Hierarchical Visual Information Processing	15
2.2 Feature Representation in the Neocortex.....	16
2.3 Cortical Columns and Image Similarity	17
2.4 HTM Foundation.....	17
2.5 Spatial Pooler in HTM.....	20
2.5 Temporal Memory (TM) Predictions	21
2.6 Approach.....	25
2.7 Memory and Similarity in Neocortex: Role of Hippocampus	26
2.8 Finding Image Similarity: Neural Mechanisms	27
2.9 Concept of KNN:	30
2.10 Feature Extraction for Image Data in KNN	31
2.11 Neocortex API KNN Applications	32
3. Theoretical Background	34
3.1 NeocortexAPI: Biological Inspiration.....	34
3.2 K-nearest neighbors (KNN) Classifier	34
3.3 K-Nearest Neighbors (KNN) in the Context of NeocortexAPI	35

3.4	Neocortical Mechanisms in KNN-Based Image Similarity	38
3.5	Applications of KNN-Based Image Similarity in NeocortexAPI	39
3.4	Steps in KNN Classification:	41
3.5	KNN Variants and Improvements for Image Classification	42
3.6	Advantages and Applications:	45
3.7	Challenges and Considerations:.....	45
4.	Methodology	48
4.1	Research Design.....	48
4.2	Data Collection Methods	49
4.3	Data Analysis Method	51
5.	Results and Discussion	54
5.1	Generate Output SDRs	54
5.2	Creating Output SDRs Images	54
5.3	Training & Test Output SDRs	55
5.4	Find Similarity via Classifier	56
5.5	Analyzing the Similarity	57
5.6	Adjusting HTM Configuration:	58
6.	Conclusion	60
	Bibliography.....	61
	Declaration of Authorship	63

List of Figures

<i>Fig 1: Machine Learning- Supervised and Unsupervised learning.....</i>	<i>12</i>
<i>Fig 2: The HTM Spatial Pooler.....</i>	<i>15</i>
<i>Fig 3: Population coding in Neural Coding.....</i>	<i>16</i>
<i>Fig 4: Pattern separation and pattern completion in neocortex.....</i>	<i>27</i>
<i>Fig 5: Simple Hebbian Learning.....</i>	<i>28</i>
<i>Fig 6: High-Level K-Nearest Neighbors.....</i>	<i>36</i>
<i>Fig 7: Understanding K-Nearest Neighbors.....</i>	<i>38</i>
<i>Fig 8: Flowchart of experiment.....</i>	<i>48</i>
<i>Fig 9: Input SDR 2D array Data.....</i>	<i>50</i>
<i>Fig 10: Creating Input SDR images.....</i>	<i>50</i>
<i>Fig 11: Input SDR images.....</i>	<i>50</i>
<i>Fig 12: Generating Output SDR via Spatial pooler.....</i>	<i>53</i>
<i>Fig 13: Creating Output SDR images.....</i>	<i>55</i>
<i>Fig 14: Training Output SDRs.....</i>	<i>56</i>
<i>Fig 15: Output Similarity prediction.....</i>	<i>56</i>
<i>Fig16: Updated HTMConfig.....</i>	<i>58</i>

List of Abbreviations

KNN	K-Nearest Neighbours
HTM	Hierarchical Temporal Memory
SDR	Sparse Distributed Representation
TM	Temporal Memory
AI	Artificial Intelligence
CNNs	Convolutional Neural Networks
IT	Inferotemporal Cortex
SP	Spatial Pooler
API	Application Programming Interface
MNIST	Modified National Institute of Standards and Technology
HOG	Histogram of Oriented Gradients
PCA	Principal Component Analysis
ANN	Approximate Nearest Neighbor
LSH	Locality-Sensitive Hashing
CBIR	Content-Based Image Retrieval
OCR	Optical Character Recognition
LDA	Linear Discriminant Analysis
1D	One Dimension
2D	Two Dimension

1. Introduction

Integrating the K-Nearest Neighbors (KNN) classifier with the Hierarchical Temporal Memory (HTM) framework is part of the exploration into similar pattern prediction. The HTM model is a temporal sequence understanding and prediction model that draws inspiration from the neocortex. By enabling HTM to represent temporal dependencies and KNN to classify patterns based on learnt data, the combination of HTM and KNN classifier can improve both pattern recognition and classification.

The K-Nearest Neighbors (KNN) classifier could be a basic, however effective machine learning calculation utilized for both classification and relapse errands. It is an instance-based, non-parametric calculation, meaning it makes choices based on the nearness of information datapoints in a highlight space without accepting any basic probability distribution of the information.

KNN could be a sort of instance-based or apathetic learning calculation because it does not build an express inner show but makes choices based on the complete preparing dataset. This documentation traces the technique, usage, and assessment of employing a KNN classifier for foreseeing comparative designs utilizing HTM's temporal memory yield.

1.1 Hierarchical Temporal Memory (HTM)

HTM is called a theoretical framework for machine learning, motivated by the structure and work of the human neocortex. It centers on understanding how neurons within the neocortex shape spatial and transient patterns, which are fundamental for human recognition and thinking.

KNN may be a straightforward, however successful instance-based machine learning calculation. It works by putting away all occurrences from the preparing information and, when displayed with a different latest information of datapoints, it does the below calculations:

- Calculates the separate between the unused data point and all preparing information focuses employing a separate metric (e.g., Euclidean and Hamming distance).
- Chooses the K closest neighbors.
- Predicts a lesson name based on the majority name of these neighbors (for classification assignments).

1.1.1 Instance-Based Learning:

KNN could be a sort of sluggish learner, meaning it does not unequivocally construct a show during the preparing stage. Instep, it stores the whole training dataset and concedes computation to classification time.

1.1.2 Distance Metric:

To classify an unused information point, KNN looks at the 'k' closest focuses from the preparing set. Its employments a remove metric like Manhattan distance, Euclidean distance, or Minkowski distance to degree nearness between data points.

1.1.3 K Value:

The "K" in KNN alludes to the number of neighbors to consider when making predictions. A little esteem of K makes the demonstrate more touchy to clamour within the information, while an expansive K smooths out the choice boundary but may lead to inclination.

1.1.4 Majority Voting:

In this, once the 'k' nearest neighbors are recognized, the calculation allots the modern information point to the course that's most visit among those neighbors (majority vote).

HTM Components

1.2 Spatial Pooler

A key element of Numenta's Hierarchical Temporal Memory (HTM) theory of machine intelligence, which simulates how the brain processes information, is the Spatial Pooler (SP). By transforming input data—such as sounds, images, or sensor readings—into sparse distributed representations (SDRs), the Spatial Pooler in HTM acts as a mechanism to recognize and represent spatial patterns.

Changes over input information into Sparse Distributed Representations (SDRs). SDRs is a binary vector whereas it where a little division of bits are dynamic, by this improves pattern recognition and resistant to noise.

1.3 Temporal memory

Modelling the brain's capacity to learn sequences and form predictions based on temporal patterns is the goal of the Temporal Memory (TM) component of Hierarchical Temporal Memory (HTM). The TM component allows an HTM system to predict and recognize sequences, capturing the context in which patterns emerge over time. It is inspired by the structure and behaviour of neurons in the neocortex.

Learns temporal groupings by capturing how inputs alter over time. TM permits the framework to create forecasts based on chronicled arrangements and transient conditions

2.2 K-Nearest Neighbors (KNN) Classifier

1.4 How KNN Works

1.4.1 Data Representation:

Each occurrence within the dataset is spoken to as a point in an n -dimension feature space, where n is the number of highlights.

1.4.2 Distance Calculation:

When a modern occasion must be classified, the calculation calculates the separate between this instance and all occasions within the preparing dataset employing a chosen distance metric.

1.4.3 Neighbor Determination:

It chooses the k occurrences within the preparing information that are closest to the modern occasion.

1.4.4 Prediction:

- Classification: The course of the modern occurrence is decided by the lion's share lesson among the k closest neighbors (larger part voting).
- Regression: The esteem of the modern occasion is anticipated by averaging all the values of the k nearest neighbors.

Problem Statement

The objective is to classify these predictions using KNN after using HTM to predict patterns in time-series data. KNN will categorize the predictions into predetermined groups according to their similarity to past patterns, while HTM will deal with learning temporal sequences.

1.5 Sparse Distributed Representation

When utilizing a KNN classifier with Hierarchical Temporal Memory (HTM), the binary vector output that is generated by HTM's Temporal Memory (TM) following the acquisition of temporal sequences is referred to as the predicted Sparse Distributed Representation (SDR). This predicted SDR is then used as input for pattern classification by the KNN classifier. A binary vector known as an SDR (Sparse Distributed Representation) has the majority of its bits set to 0 and only a small portion of its bits active (set to 1). This sparse activation pattern is robust to noise and effectively encodes information, capturing pertinent input features.

Important characteristics of SDRs:

- **Sparsity:** The effects of noise are lessened because only a small number of bits are active.
- **High Dimensionality:** Many different patterns are possible because SDRs are usually high-dimensional (2048 bits or more).
- **Similarity Measure:** The Hamming distance, which counts the number of bits that separate two binary vectors, is frequently used to calculate how similar two SDRs are to one another.

1.6 Required Technologies

1.6.1 Data Analysis:

Data analysis is essential to extracting the insights and valuable information from the data.

The process of cleaning and transforming data so that it is prepared for decision-making is known as data analysis. It is a crucial procedure used by all contemporary corporate entities.

1.6.2 Machine Learning:

Machine learning has become incredibly popular and widely used over the years in a variety of industries, including marketing and information technology.

As a subset of artificial intelligence, machine learning uses various statistical models to predict features without the need for human programming. Although they are capable of a wide range of tasks, they rely on many patterns.

Machine learning algorithms are developed by using training data, which is a sample of data, to build a model.

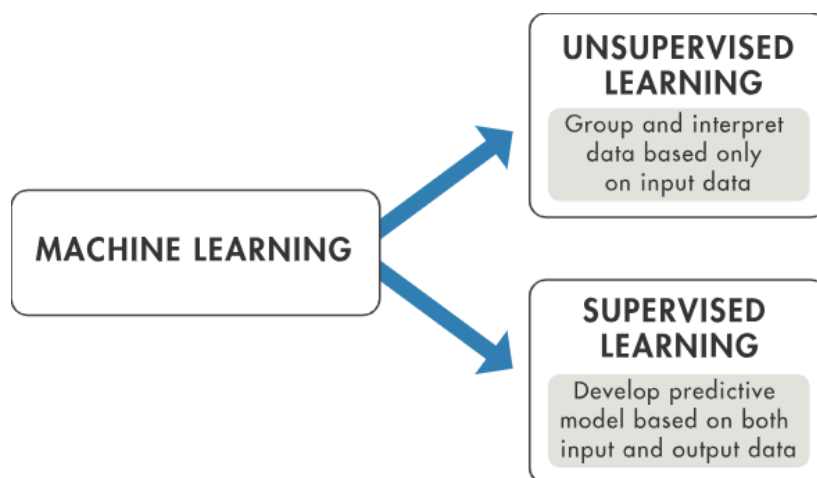


Fig 1: Machine Learning- Supervised and Unsupervised learning

Machine learning can perform any type of analysis, including exploratory, predictive, descriptive, and prescriptive methods.

The machine learning algorithms used primarily determine these analyses.

1.6.3 Supervised and Unsupervised Learning:

Supervised learning and unsupervised learning are the two primary machine learning methodologies that are pertinent to this thesis.

In order to find predictions, supervised learning uses training data that has already been labelled; the dataset must be divided for testing. Without any training, unsupervised learning is effective.

2. Literature Review

Theory and Concepts of Image Similarity in the Neocortex.

The region of the brain is in charge of the higher-order processes like sensory processing, perception, and visual recognition of the data is called as the neocortex in this concept. Deep insights into artificial intelligence (AI) and neuroscience can be gained by comprehending how the neocortex processes image similarity. The way the brain organizes and retrieves visual memories, how it processes visual stimuli to determine image similarity, and how this knowledge can inform computer vision models like convolutional neural networks (CNNs) are of particular interest to neuroscientists and AI researchers.

Jeff Hawkins' interest in comprehending how the neocortex facilitates human intelligence is where the idea of Hierarchical Temporal Memory (HTM) first emerged in neuroscience. Hawkins is a neuroscientist and computer scientist who first became well-known in the 1980s for creating the PalmPilot. However, he quickly focused on studying the brain. His objective was to create a theory that would explain the hierarchical, sequential way in which the brain processes information. By outlining a neocortical model centered on pattern recognition and sequence learning, Hawkins' 2004 book *On Intelligence* established the groundwork for HTM theory.

Hawkins established Numenta in 2005 to conduct research on brain-inspired computing, shortly after publishing *On Intelligence*. Hawkins and his colleagues at Numenta created and improved HTM algorithms, which were largely inspired by the columnar organization and sparse distributed representations (SDRs) of the neocortex, which is a biological structure. For HTM to learn sequences and make predictions, they concentrated on capturing crucial characteristics like spatial and temporal memory. This represented a change from conventional machine learning models, which were mainly statistical in nature rather than biologically motivated.

As new tools and algorithms were created over time, such as the open-source Numenta Platform for Intelligent Computing (NuPIC), HTM changed as well. Numenta published studies describing developments in temporal memory, spatial pooling, and sequence prediction as well as other aspects of HTM theory. HTM became well-known for its anomaly detection capabilities, particularly in applications that call for time-series analysis and unsupervised learning, like tracking down equipment malfunctions and identifying odd activity in data streams.

HTM research has broadened in recent years to tackle issues like scalability, integration with conventional AI techniques, and applications in domains like cybersecurity and the Internet of Things. Despite being a subset of artificial intelligence, HTM is regarded as a groundbreaking

method of brain-inspired computing, and Numenta is currently conducting research to better understand the neocortex and increase the practical applications of HTM.

It is believed that neural mechanisms that facilitate pattern recognition, object identification, and memory recall process image similarity in the neocortex. In order to evaluate similarity, the neocortex—the area of the brain in charge of higher-order processes like sensory perception and cognition—processes visual input by recognizing both common and distinctive characteristics of images. Some of the fundamental theories and ideas are listed below:

2.1 The Neocortex and Hierarchical Visual Information Processing

Similar to deep learning models, the neocortex uses a hierarchical processing structure when processing visual information. It is organized into layers, each of which is in charge of representing sensory inputs in increasingly/incremental abstract ways in the neocortex: The first cortical area to process visual information from the eyes is the primary visual cortex (V1). Basic characteristics like edges, lines, and orientation are extracted.

- The V2, V3, and V4 higher visual areas: By identifying more intricate patterns like shapes, textures, and colors, these regions expand on the outputs from V1.
- Inferotemporal cortex (IT): Responsible for identifying objects and assessing how similar they are. High-level abstract features, such as complete objects or faces, are processed by it.

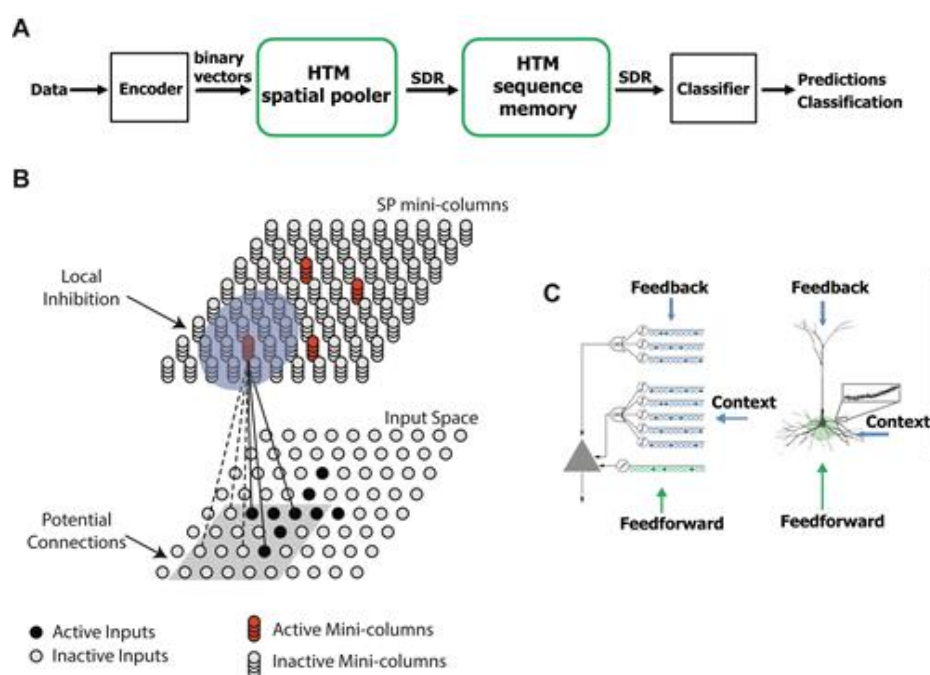


Fig 2: The HTM Spatial Pooler—A Neocortical Algorithm for Online Sparse Distributed Coding

This hierarchy suggests that the brain compares feature representations rather than raw sensory data, such as pixels in an image, when determining similarity. In the end, a feature space is created where image similarity is calculated based on shared patterns, combining low-level features like edges (in V1) into more intricate patterns (in higher visual regions).

2.2 Feature Representation in the Neocortex

Distributed representations are used by the neocortex to process visual information. It represents an image as a collection of numerous features dispersed throughout various neurons rather than encoding it as a single point.

- **Sparse coding:** It is believed that neurons in the neocortex employ sparse coding, which means that only a tiny portion of neurons react strongly to a given stimulus. This enables the brain to use only the most pertinent features to represent images in a highly efficient manner.
- **Population coding:** The neocortex most likely employs this technique, in which a large number of neurons collectively represent an idea or picture. Different objects or images may be associated with different patterns of neuronal activation, while similar images may be associated with similar patterns of activation.

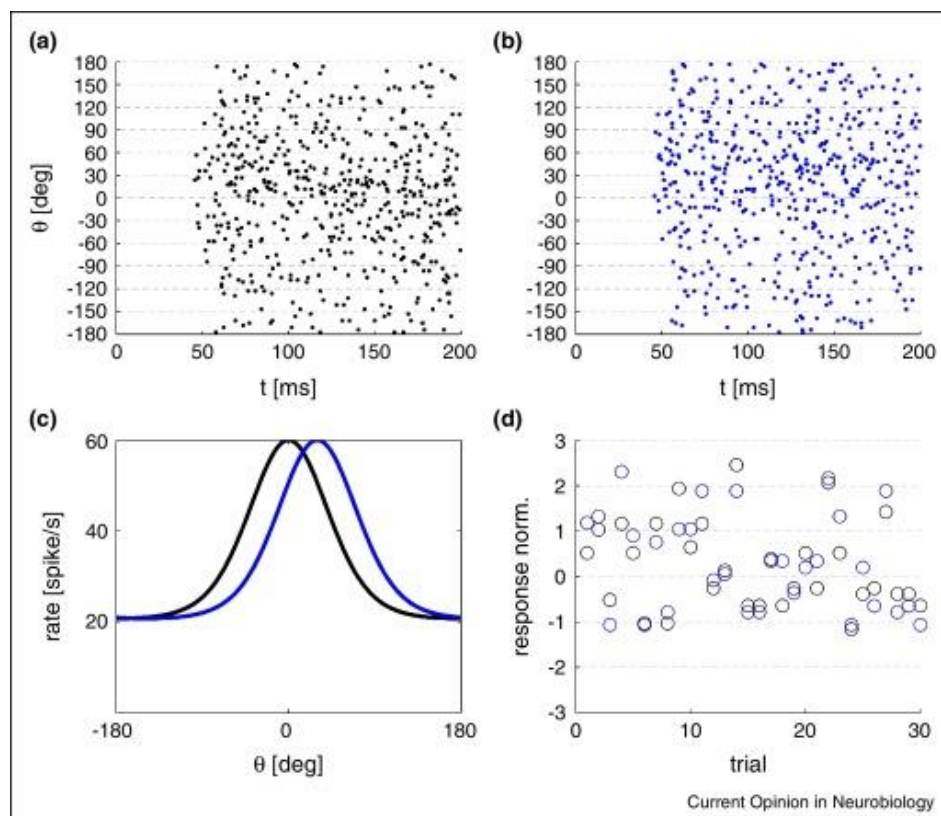


Fig3: Population coding in Neural Coding

Analogous of these concepts can be found in machine learning, where models such as CNNs and autoencoders create feature vectors to represent images. Similar images in the latent space produce similar vectors.

2.3 Cortical Columns and Image Similarity

The arrangement of neurons into cortical columns, which are believed to process various facets of the same sensory input, is one of the basic principles of the neocortex:

- **Mini-columns:** These are tiny clusters of neurons that react to a particular input characteristic, like an edge in a certain orientation. Different facets of the same input are processed by several mini-columns.
- **Macro-columns:** These bigger structures help identify whole objects or more intricate patterns by combining data from numerous mini-columns. They can be viewed as units that process and compare more and more comprehensive object information in dataset.

Overlapping columns of neurons are probably reacting to both inputs in similar ways when the brain detects similarities between two images. The brain may interpret two images as similar if they have similar contours or shapes, for instance, because the same mini-columns may be triggered in response to both.

2.4 HTM Foundation

A computational framework called Hierarchical Temporal Memory (HTM) was created to process, identify, and forecast patterns in sequential data. It was inspired by the functions and structure of the human neocortex. HTM has its roots in neuroscience, specifically in the way the neocortex processes sensory data in the brain. Information flows through layers and columns in the neocortex's hierarchical structure, making it possible to recognize patterns that get more and more abstract. The hierarchy of "mini-columns" and "macro-columns" that make up the foundation of HTM's architecture replicates this idea. These columns are collections of simulated neurons that cooperate to learn sequences, identify spatial patterns, and make predictions about the future. HTM seeks to capture some of the most potent facets of human intelligence, especially the capacity to identify temporal sequences and predict data changes, by simulating these fundamental neocortical features.

Sparse Distributed Representations (SDRs) are a key component of HTM. Only a small portion of neurons are active at any given time in the human brain due to the distributed encoding of information, which results in extremely effective and noise-resistant data processing. By generating sparse representations of input data—in which only a tiny subset of columns become

active in response to a given input—HTM reproduces this. Because of its sparseness, HTM can reliably detect patterns even when there is noise or incomplete data. Each bit in SDRs, which are binary vectors, indicates whether a mini-column is active or inactive. SDRs can compactly represent large amounts of information because only a small percentage of their bits are active. This allows the system to effectively distinguish between various inputs and identify similarities between them.

Temporal memory, which enables the system to learn and anticipate sequences, is another essential part of HTM. The way neurons in the neocortex connect based on time sequences serves as the model for temporal memory. HTM allows neurons in a mini column to go into a "predictive state," where they are ready to become if a particular pattern is anticipated based on past inputs. By creating connections, or synapses, between neurons that often activate consecutively, this prediction is realized. The HTM network can remember and predict the order of patterns in a sequence thanks to these connections. After learning a series of patterns, HTM can use the temporal context that previous inputs have provided to predict the next likely input. Tasks like time-series prediction, where knowing the order and timing of events is essential, benefit from this capability.

In HTM, synaptic plasticity simulates how experience shapes and fortifies connections between neurons in the brain. Each neuron in HTM possesses several segments on its dendrites, which resemble branches and can connect to other neurons to form synapses. According to the Hebbian learning principle, "cells that fire together, wire together," connections between active neurons become stronger when a particular pattern is repeated. Because of its adaptability, HTM can gradually pick up new sequences and adjust its connections in response to shifting patterns in the data. With the ability to dynamically add or remove synapses, HTM offers a degree of adaptability akin to that of biological neural networks. Because unexpected inputs that don't match any learned sequence or pattern cause the system to register them as anomalies, this mechanism also enables HTM to perform anomaly detection.

Because of its hierarchical structure, HTM allows for the combination of simpler patterns found at lower levels of the hierarchy to create more complex patterns at higher levels. The way the neocortex processes information at various levels of abstraction is like this hierarchy. In the visual system, for instance, simple edges and lines are processed by lower layers of the cortex, while more complex shapes and objects are created by higher layers combining these features. Simple, recurrent patterns in input data are identified by the model's lower layers, which then relay these representations up the hierarchy. These basic patterns are combined by higher layers to identify more intricate sequences or higher-order abstractions. By decomposing complex temporal data, like time-series information, into simpler parts and then reassembling them in a meaningful manner, this hierarchical structure enables HTM to comprehend the data.

One of HTM's primary features is its predictiveness, which makes it ideal for sequence learning and anomaly detection applications. The temporal memory and hierarchical structure of HTM enable the system to predict future inputs based on learned patterns, which is the foundation of its predictive power. For example, in anomaly detection, HTM records a deviation from the expected sequence as an anomaly by comparing incoming data to its learned sequences. This

is helpful in situations where abrupt changes in sensor data could point to a malfunction, such as when monitoring the health of equipment. Because HTM's predictions are essentially probabilistic—that is, it anticipates patterns that resemble previously observed sequences rather than precise repetitions—it can handle naturally variable real-world data.

In conclusion, HTM is a distinct method of machine learning that reflects many facets of human intelligence, especially in its emphasis on prediction and sequence learning. HTM systems can identify and forecast patterns in data, even when there is noise or incomplete information, by combining temporal memory, hierarchical structure, and sparse distributed representations. HTM is ideally suited for real-world applications due to its biologically inspired architecture, particularly those involving time-series data or any field where anomaly detection and pattern recognition are crucial. Despite being a developing technology, HTM presents a promising path toward artificial intelligence that is more akin to humans thanks to its solid neuroscience foundation.

Biological Feature	HTM Model Component	Functionality
Cortical Columns	HTM Columns (Mini-columns)	Process and recognize spatial patterns
Neurons and Synapses	Neurons with Synaptic Connections	Learning and storage of temporal sequences
Hierarchy	Multi-Layered Network Structure	Multi-level pattern recognition
Sparse Distributed Representation	Sparse Binary Vectors (SDRs)	Efficient, noise-resistant encoding of patterns
Temporal Memory	Sequence Learning Mechanism	Prediction based on learned sequences
Predictive Coding	Neuron Prediction States	Anticipation of input, anomaly detection

HTM Configuration Parameters:

POTENTIAL_RADIUS: It defines the radius in numbers of input cells which are visible to column cells. It is really important to choose these values, so that every input neuron is connected to at least one of a single column. For instance, if the input has 50000 bits and column's topology is 500, then you should choose a value larger than $50000/500 > 100$.

GLOBAL_INHIBITION: If its TRUE then global inhibition algorithm will be used and if it is FALSE then local inhibition algorithm will be used.

LOCAL_AREA_DENSITY: Density of all active columns inside of the local inhibition radius. If it is set on value < 0 , then explicit numbers of active columns (**NUM_ACTIVE_COLUMNS_PER_INH_AREA**) will be in usage.

POTENTIAL_PCT: It defines the percent of the inputs within potential radius, which should be connected to the columns.

INHIBITION_RADIUS: It defines the neighbouring radius of a column.

STIMULUS_THRESHOLD: One mini-column is in active if it's overlap value exceeds overlap threshold of the connected synapses.

NUM_ACTIVE_COLUMNS_PER_INH_AREA: Here is an alternate way to control the densities of active columns. If these values are specified, then **LOCAL_AREA_DENSITY** should be less than 0.

SYN_PERM_CONNECTED: It defines Connected Permanence Threshold, which is known as float value, it must be exceeded to be declare as synapse are connected.

SYN_PERM_ACTIVE_INC: Incremental steps of connected synapse during the learning process

SYN_PERM_INACTIVE_DEC: Decremental steps of synapse permanence value within every inactive cycle. It is defined how fast the NeoCortex will be forgetting learned patterns.

MAX_BOOST: Maximum boosting factor of a column.

DUTY_CYCLE_PERIOD: It defines the Number of iterations. The period used to calculate the duty cycles. Higher the value longer to respond to changes in boost. Shorter the values more unstable and likely to oscillate.

2.5 Spatial Pooler in HTM

The data's spatial properties are handled by Spatial Pooler, which converts input patterns into sparse binary representations. Consequently, it is in charge of encoding sparse and stable representations of input data. To capture temporal dependencies and generate predictions, the TM expands upon these representations. The following guidelines govern how the Spatial Pooler functions:

- **Input Encoding:** An encoder first converts input data into a binary vector of 1s and 0s. Relevant input features must be captured by this encoding.
- **Columns and Connections:** The Spatial Pooler is made up of a number of "columns," each of which can be viewed as a collection of brain cells that are connected to a subset of input bits. We refer to these connections as "synapses."

- **Synapse Permanence:** The strength or permanence of the connection is indicated by the "permanence" value assigned to each synapse. A synapse becomes "connected" and contributes to the activation of the column if its permanence surpasses a particular threshold.
- **Overlap Score:** The number of active (1) inputs connected to a given column calculates the "overlap score". The input activates columns with higher overlap scores more.
- **Inhibition:** The Spatial Pooler only chooses a subset of columns with the highest overlap scores to guarantee sparsity. Sparse representations are produced by this process, also known as "k-winners-take-all" or inhibition, which makes sure that only a limited number of columns become active for a given input.
- **Learning and Adaptation:** To enhance the accuracy and stability of its representation, the Spatial Pooler gradually modifies synapse permanence. The synapses in columns with a high overlap score are strengthened, whereas those in other columns may become weaker.

Sparse Distributed Representation (SDR) is the output of Spatial Pooler, A binary vector known as a Sparse Distributed Representation (SDR), in which only a small percentage of bits are active (i.e. 1s) at any given time. SDRs offer several advantageous characteristics:

Robustness to Noise: The SDR is noise-resistant because small changes in the input data only slightly alter it.

Effective Storage: SDRs' sparse design makes it possible to use memory effectively.

High Capacity: Because SDRs are sparse, they can hold a lot of unique patterns.

Since the Temporal Memory uses these representations to learn sequences and make predictions, the Spatial Pooler's capacity to produce sparse, stable representations is essential to the HTM system. The Spatial Pooler improves temporal learning efficiency by emphasizing spatial patterns, laying a solid basis for identifying and forecasting temporal sequences over time.

2.5 Temporal Memory (TM) Predictions

In HTM, the next likely SDR can be predicted by the Temporal Memory (TM) once it has learned a series of inputs. Based on past patterns, the TM accomplishes this by turning on cells that are most likely to be involved in the subsequent step of the sequence. The system's prediction of the subsequent input in the sequence is represented by the predicted SDR, a binary vector output from the Temporal Memory that records the temporal dependencies that were learned in the input sequence.

Sequence Learning: Sequences of sparse distributed representations (SDRs) produced by the Spatial Pooler are learned by Temporal Memory. The TM is able to predict what might happen next by learning the transitions between SDRs when given a series of inputs. This is comparable to how the brain anticipates future events and recognizes sequences.

Mini-columns and Cells: TM is based on a structure of mini-columns made up of cells. While individual cells within a column represent various contexts for a feature, each mini-column represents a specific feature or input pattern. For instance, depending on what came before in the sequence, different cells within the same mini-column may activate in response to a specific sensory input.

States of Activity and Predictiveness: Upon receiving an input, specific cells in columns turn active if the input matches previously learned patterns. If cells have learned to predict the current input from previous sequences, they go into a predictive state. The learned sequence is reinforced when an input later activates a cell in a predictive state.

Contextual Prediction: TM can generate a predictive context for every input pattern by linking particular cells in columns to earlier inputs. This enables TM to distinguish between identical inputs that appear in various sequences. For example, the word "apple" might elicit distinct predictions depending on whether it comes after "red" or "green."

2.5.1 Using the Predicted SDR in KNN Classification

A KNN classifier can be used to further categorize the predicted SDR into one of several established groups after HTM has generated it. In order to find the closest patterns in a historical dataset, the predicted SDR is treated as the query vector.

2.5.2 Overview of the KNN Classifier

A non-parametric algorithm called the KNN classifier uses known instances from the training set to classify data points.

- Calculating the distances between the new point (the predicted SDR) and every earlier point in the dataset is how it operates.
- Using the computed distances, choose the K closest neighbors.
- Using the K nearest neighbours' average (regression) or majority vote (classification) to classify the new point.

2.5.3 SDR Prediction and Classification Workflow

Step 1: Generate Predicted SDR

The HTM Temporal Memory learns a series of inputs and then produces a predicted SDR. The system's best estimate of the next input's appearance is represented by this SDR.

```
predicted_sdr = tm.getPredictiveCells() # Get predicted SDR from HTM's Temporal Memory
```

Step 2: Calculate Distance (Hamming Distance)

Since SDRs are binary vectors, we usually compare the predicted SDR with known SDRs using the Hamming distance. The number of positions where the corresponding bits differ between two binary vectors is known as the Hamming distance.

For instance, with two SDRs:

- $SDR1 = [1, 0, 0, 1, 0]$
- $SDR2 = [0, 0, 1, 1, 0]$

Given that the vectors differ at the first and third positions, the Hamming distance would be 2. The following is how to calculate the Hamming distance:

```
import numpy as np
def hamming_distance(sdr1, sdr2):
    return np.sum(sdr1 != sdr2) # Count the number of differing bits
```

Step 3: Train KNN on Historical SDRs

We then use previously created SDRs (from historical input data) and their matching labels to train the KNN classifier. The classes or categories that the KNN model is supposed to learn are represented by these labels.

```
from sklearn.neighbors import KNeighborsClassifier

# Assume we have historical SDRs and their corresponding labels
historical_sdrs = ... # Array of historical SDRs (training data)
labels = ... # Corresponding class labels

# Initialize KNN with Hamming distance as the metric
knn = KNeighborsClassifier(n_neighbors=5, metric='hamming')

# Train KNN
knn.fit(historical_sdrs, labels)
```

Step 4: Classify the Predicted SDR

The predicted SDR from HTM can be classified using the KNN model once it has been trained. The classifier will use majority voting to assign a class label to the predicted SDR after determining the K nearest neighbors (based on Hamming distance).

```
# Use KNN to classify the predicted SDR
predicted_class = knn.predict([predicted_sdr])
print(f'Predicted class: {predicted_class}')
```

2.5.4 Predicted SDR Example in KNN Classification

This is a thorough example that demonstrates the procedure:

2.5.4.1 Explain SDRs and Their Labels

Assume that the following SDRs, along with the labels that go with them, represent historical data.

```
# Example SDRs (5-bit binary vectors for simplicity)
historical_sdrs = np.array([
    [1, 0, 0, 1, 0],
    [0, 1, 1, 0, 0],
    [1, 1, 0, 1, 1],
    [0, 0, 1, 1, 0],
    [1, 0, 1, 0, 1]
])

# Example class labels
labels = np.array([0, 1, 0, 1, 0])
```

2.5.4.2 Train KNN Classifier

Using the previously stated historical SDRs and their labels, we train a KNN classifier:

```
knn = KNeighborsClassifier(n_neighbors=3, metric='hamming')
knn.fit(historical_sdrs, labels)
```

2.5.5 Classify a New Predicted SDR

Let's say the following SDR is predicted by the **HTM Temporal Memory**:

```
predicted_sdr = np.array([1, 0, 0, 1, 0]) # This SDR comes from HTM's prediction
```

With the trained KNN model, we can categorize this anticipated SDR:

```
predicted_class = knn.predict([predicted_sdr])
print(f"Predicted class: {predicted_class[0]}")
```

Output:

```
Predicted class: 0
```

Based on its closest neighbors in the training data, the predicted SDR is categorized as being in class 0.

2.6 Approach

2.6.1 Data Preprocessing

The first step in the process is getting the data ready for the HTM model.

- **Time-Series Data:** Any type of time-series data, such as stock prices, sensor readings, etc., can be used as input.
- **Feature Scaling:** To guarantee consistent distance calculations, you might need to use feature scaling, also known as normalization, depending on the type of data.
- **Encoding:** For the Spatial Pooler to process data, it must be encoded.

For instance:

- **Scalar Encoder:** For numerical data that is continuous. For categorical data, use a category encoder.
- **Date-Time Encoder:** For cyclic data, such as day or month, and timestamps.

2.6.2 HTM Temporal Learning

Data is fed into HTM for temporal learning after it has been pre-processed.

2.6.2.1 Spatial Pooling:

Sparse Distributed Representations (SDRs) are created from input data by the Spatial Pooler.

SDRs are binary vectors in which a tiny portion of the total vector is represented by each active bit. These SDRs ignore noise or extraneous information while capturing the input's key characteristics.

2.6.2.2 Temporal Memory

Sequences are learned by Temporal Memory (TM) from the SDRs produced by the Spatial Pooler.

- By using historical sequences to predict the next input, the TM monitors temporal dependencies in the data.
- A predicted SDR, or the next likely pattern in the sequence, is the TM's output.

2.6.3 KNN for Pattern Classification

A KNN classifier is used to classify the predicted patterns after HTM generates a prediction.

2.6.3.1 Distance Metric:

- It is necessary to compute the separation between the anticipated SDR and SDRs that have already been observed.
- The Hamming distance is frequently employed because SDRs are binary vectors. This distance metric, which is effective for binary data, determines how many bits differ between two SDRs.

2.6.3.2 KNN Classification

- The KNN classifier uses the calculated distance to find the K nearest neighbors.
- The predicted SDR is given the majority class label among its K nearest neighbors based on the majority voting principle.
- Through this procedure, KNN can assign the anticipated sequence to one of several pre-established groups.

2.7 Memory and Similarity in Neocortex: Role of Hippocampus

The hippocampus is crucial for storing and retrieving memories of images, whereas the neocortex is necessary for processing and comparing visual information. The hippocampus encodes and retrieves memories of similar images using processes like pattern separation and pattern completion:

- **Pattern separation:** The brain uses this mechanism to discern between images that are very similar but not the same. The brain uses pattern separation, which is thought to be mediated by the hippocampus, to distinguish between images even when they share many characteristics.
- **Pattern completion:** In contrast, even with incomplete information, the brain is able to recall comparable images from memory. The brain "fills in the gaps" to identify similar images in a process called pattern completion.

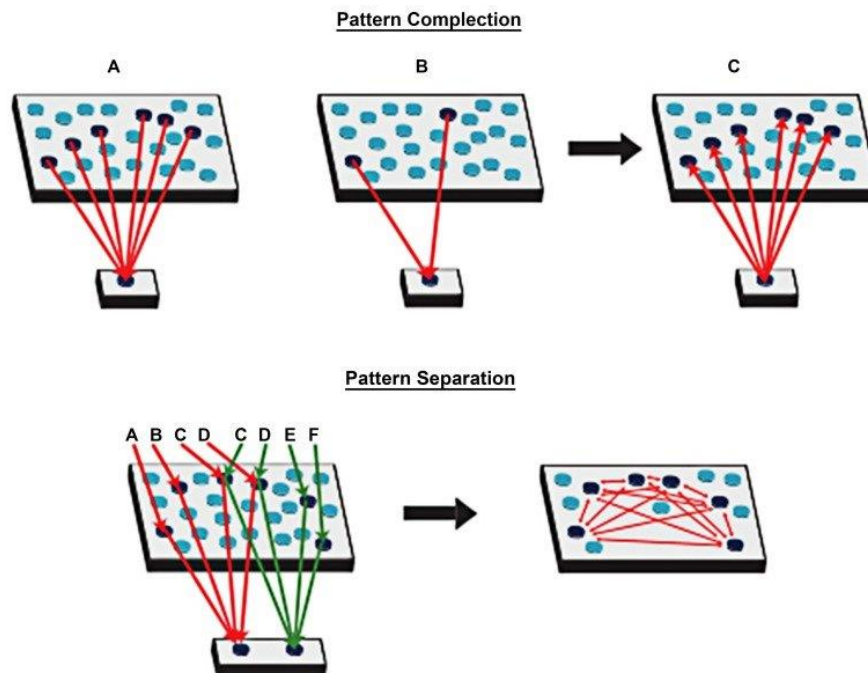


Fig4: Pattern separation and pattern completion in neocortex

2.8 Finding Image Similarity: Neural Mechanisms

The brain's capacity to identify and compare similar images has been attributed to a number of neural processes. Among the crucial procedures are:

2.8.1 Receptive Fields

Each neocortical neuron has a receptive field, which means it reacts to a particular area of the visual field. Higher-level neurons integrate information from a greater area of the image because they have larger receptive fields. Neurons with overlapping receptive fields may fire similarly when two images are compared, indicating that the images are similar.

- Simple features like edges or textures are compared by low-level neurons (like in V1).
- Complex shapes and objects are compared by higher-level neurons, such as those found in IT.

2.8.2 Temporal and Spatial Coding

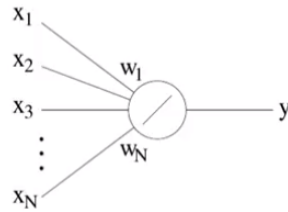
Temporal and spatial coding may also be involved in image similarity in the neocortex:

- Relationships between features can be encoded using temporal coding, which is the timing of neuronal firing. Parts of the same object may be represented by neurons that fire together.
- Spatial coding refers to the location of neuronal firing in the brain. Similar features may be more likely to be represented by neurons that are close to one another.

2.8.3 Hebbian Learning

The neocortex strengthens the connections between neurons that often fire in response to similar stimuli by using Hebbian learning ("cells that fire together, wire together"). As a result, images with similar features become more strongly associated over time, which facilitates retrieval and comparison.

Linear Unit:



$$y = \mathbf{w}^T \mathbf{x}. \quad (\text{linear unit}) \quad (1)$$

$$\Delta w_i = \eta \frac{1}{M} \sum_{\mu} y^{\mu} x_i^{\mu} =: \eta \langle y x_i \rangle_{\mu} \quad (2)$$

$$\iff \Delta \mathbf{w} = \eta \langle y \mathbf{x} \rangle_{\mu} = \eta \langle (\mathbf{w}^T \mathbf{x}) \mathbf{x} \rangle_{\mu}. \quad (3)$$

Fig5: Simple Hebbian Learning

Neocortical Theories in Image Similarity Algorithms: AI Applications

More biologically inspired AI systems for image similarity are currently being designed using many of the neocortical principles. For instance:

2.8.4 Convolutional Neural Networks (CNNs) and Deep Learning Models

With convolutional layers imitating the function of V1 and higher layers imitating later cortical areas, CNNs are directly inspired by the structure of the visual cortex. CNNs extract features

at multiple scales, similar to the neocortex. Early layers identify edges, while later layers identify more abstract patterns (like objects).

Similar to how the brain compares various visual inputs, CNNs produce feature embeddings for image similarity tasks that can be compared using distance metrics like cosine similarity or Euclidean distance.

2.8.5 Hierarchical Temporal Memory (HTM)

The neocortex served as the inspiration for Jeff Hawkins' HTM, which simulates the brain's capacity for pattern recognition and prediction. HTM systems try to mimic the temporal and hierarchical processing found in the neocortex in image similarity tasks.

2.8.6 Capsule Networks

The goal of Capsule Networks, another biologically inspired architecture, is to more closely simulate how the brain encodes spatial relationships between image features. By capturing more intricate part-whole relationships in images, these networks aim to outperform CNNs.

2.8.7 Neocortical Theories of Invariance and Generalization

Invariance—the capacity to identify objects in spite of variations in scale, rotation, illumination, or perspective—is a crucial aspect of human visual recognition. By generalizing patterns across various transformations, the neocortex achieves this.

- Neural mechanisms of invariance: Higher cortical regions, like the inferotemporal cortex (IT), have neurons that react similarly to variations of the same object but are less sensitive to slight changes in the image. This enables the brain to identify that an object that has been scaled or rotated is still the same object.
- Generalization in AI models: Siamese networks and triplet loss networks are two examples of contemporary AI systems that are trained to learn invariant representations of objects, drawing inspiration from the neocortex. Similar to the human brain, these networks are made to compare images based on their key characteristics while disregarding unimportant variations.

2.9 Concept of KNN:

The k-nearest neighbors (KNN) algorithm is a simple yet powerful instance-based learning method for classification and regression problems. It makes use of the proximity principle, which says that the majority class of a sample's k-nearest neighbors in the feature space determines its classification. The main advantages of KNN are its simplicity, ease of implementation, and effectiveness in a variety of domains.

2.9.1 KNN Foundations

To find the closest neighbors, KNN uses distance metrics like the Manhattan, Minkowski, or Euclidean distances. Important things to think about are:

Choice of k: Performance can be greatly impacted by the choice of the ideal number of neighbors.

Distance metric: The decision boundary's shape is affected by the metric selection. The curse of dimensionality states that distance concentration causes KNN performance to decline in high-dimensional spaces.

2.9.2 KNN in Neocortex API

Neocortex API, developed by Numenta, implements KNN as part of its machine learning capabilities, leveraging the principles of hierarchical temporal memory (HTM). This allows for a unique approach to anomaly detection and pattern recognition in time-series data. Implementation Features are as following:

- **Data preprocessing:** Before using KNN, the Neocortex API facilitates effective feature extraction and normalization through a variety of preprocessing techniques.
- **Scalability:** The API makes it possible to scale, which makes it appropriate for big datasets that are frequently used in real-time applications.
- **Integration with HTM:** KNN and HTM work together to improve the algorithm's capacity to recognize temporal patterns, which is especially helpful for applications like anomaly detection and predictive maintenance.

2.9.3 KNN for Image Classification

Since the earliest days of pattern recognition research, KNN has been used to classify image data. KNN's fundamental tenet is that pictures with comparable pixel intensities or

characteristics ought to be in the same class. KNN categorizes new images according to the closest labelled examples in this space, treating images as high-dimensional data points with each pixel or feature functioning as a dimension.

2.9.4 Basic KNN for Image Data:

- Originally, raw pixel values were usually used as features in KNN implementations for image classification. For example, KNN is used with Euclidean distance as the metric in the MNIST digit classification problem, where each image is treated as a vector of pixel intensities.
- KNN was first presented by Hart, Duda, and Stork (2001) as a baseline classifier for pattern recognition, proving that it could successfully classify image data without the use of intricate models.

They did point out, though, that the dimensionality of the data and the distance metric selection have a significant impact on KNN's performance.

2.9.5 Distance Metrics in KNN for Image Data:

- For image data, the standard KNN algorithm usually uses Euclidean distance. To enhance performance, alternative distance metrics have been investigated.
- Zhao et al. (2010) examined the effects of various distance metrics on the performance of KNN in image recognition tasks, demonstrating that the choice of distance metric can significantly affect the classifier's accuracy. Cosine similarity, Manhattan distance, and Mahalanobis distance have been found to be effective in some applications where Euclidean distance might well fail, especially when the data is not uniformly distributed in the feature space.

2.10 Feature Extraction for Image Data in KNN

When applied directly to high-dimensional image data, raw pixel-based KNN is ineffective and less accurate due to the curse of dimensionality. This has been addressed by converting images into a lower-dimensional space where KNN can function more efficiently using a variety of feature extraction techniques.

2.10.1 Histogram of Oriented Gradients (HOG):

HOG, which records the distribution of gradient orientations in an image, was first presented by Dalal and Triggs (2005). HOG features have been used in conjunction with KNN for image classification tasks and are commonly utilized in object detection. Compared to using raw pixel values, KNN with HOG features enables better discrimination between various object classes.

2.10.2 Scale-Invariant Feature Transform (SIFT):

SIFT, which was created by Lowe (1999), identifies and characterizes local features in pictures that are independent of rotation and scale. Since KNNs offer reliable local descriptors of images, they have been successfully used to classify images based on SIFT features. Specifically, sets of SIFT features have been transformed into a format that KNN can classify using the Bag of Visual Words (BoVW) model.

2.10.3 Principal Component Analysis (PCA):

Jolliffe (1986) presented PCA as a method for reducing dimensionality. Prior to training a KNN classifier, PCA is frequently used to reduce the number of features (pixels) in image classification tasks. In order to classify faces in a lower-dimensional subspace, Turk and Pentland (1991) employed PCA to derive Eigenfaces for face recognition, where KNN produced notable speedups and increased accuracy.

2.10.4 Deep Learning Features:

Since deep learning has emerged, scientists have taken high-level features from convolutional neural networks (CNNs) that have already been trained and fed them into KNNs. Using features taken from deep CNNs as input for a KNN classifier, Razavian et al. (2014) showed that they could achieve competitive results with much simpler models.

2.11 Neocortex API KNN Applications

2.11.1 Anomaly Detection

Studies show that KNN works well for finding anomalies in time-series data, especially when combined with HTM. Both temporal and spatial pattern recognition are made possible by this dual strategy.

2.11.2 Pattern Recognition

Because of its simplicity, KNN is perfect for preliminary testing in pattern recognition tasks. To create baseline results, its performance is frequently compared to more intricate algorithms.

2.11.3 Real-World Case Studies

The effectiveness of KNN in the Neocortex API framework has been reported in a number of studies:

Predictive Maintenance: By detecting departures from learned normal behaviour, KNN has been demonstrated in industrial applications to accurately forecast equipment failures.

Healthcare: KNN's usefulness in personalized medicine has been demonstrated by its application in classifying patient data according to past trends.

3. Theoretical Background

KNN Classifier for Image Similarity in NeocortexAPI is a class of machine learning models and APIs that draw inspiration from how the human neocortex functions. Especially for tasks like image recognition, classification, and similarity, these models seek to mimic the way the neocortex processes information. Using biologically inspired mechanisms to handle image similarity and retrieval tasks, the K-Nearest Neighbors (KNN) classifier is applied within the NeocortexAPI framework.

The theory underlying the use of KNN for image similarity in NeocortexAPI is broken down as follows:

3.1 NeocortexAPI: Biological Inspiration

For cognitive tasks like vision, the NeocortexAPI is a neural-inspired framework that takes inspiration from the distributed and hierarchical structure of the human neocortex. It focuses on the following ideas and imitates the neocortical cortical columns and hierarchical feature extraction mechanisms:

- **Hierarchical processing:** NeocortexAPI extracts features at progressively higher levels of abstraction by processing visual data in layers, just like the neocortex. Simple patterns, like edges, are the focus of the lower layers, while more complex patterns, like objects, are captured by the higher layers.
- **Sparse coding:** NeocortexAPI seeks to efficiently encode images by representing them with only the most pertinent features. Similar to how the neocortex represents visual stimuli using constrained neuronal resources, this sparse coding technique lowers the dimensionality and redundancy of data.
- **Pattern recognition:** Similar to the neocortex's invariance and generalization processes, NeocortexAPI is made to identify and extrapolate patterns across variations. These enable the model to recognize similarities between photos even when there are changes in scale, lighting, or orientation.

3.2 K-nearest neighbors (KNN) Classifier

The main concept of KNN is that it uses the majority class of its k nearest neighbors in the feature space to classify a new data point.

- "K" stands for the number of neighbors considered.

- "Nearest" is found using a distance metric, usually the Euclidean distance.
- The new data point is categorized by being placed in the most frequent class among its K closest neighbors.

For instance:

Consider a 2D space where points are divided into two classes, such as blue and red. The KNN algorithm assigns a new point to the class that most of its nearest K points belong to after calculating its distance to every other point.

KNN's training procedure differs significantly from that of many other algorithms (e.g., neural networks, SVMs, or decision trees). KNN is referred to as a lazy learner, which means:

- Absence of an explicit training phase: The KNN algorithm lacks a true training phase. The training data is stored in memory as part of the training process.
- Prediction during query: KNN determines the K nearest neighbors, calculates the distances between the query point and every point in the training data, and then classifies the query point based on the majority vote when a new query (test) instance is supplied.

3.3 K-Nearest Neighbors (KNN) in the Context of NeocortexAPI

One non-parametric technique for classification and retrieval that can be used for image similarity tasks is the K-Nearest Neighbors (KNN) algorithm. KNN functions over the feature space produced by a hierarchical model with biological inspiration when paired with NeocortexAPI. This is how it operates:

3.3.1 Feature Extraction and Representation

- NeocortexAPI's image processing begins with feature vector extraction. These feature vectors, which are structured similarly to how the neocortex arranges visual data, represent the image's visual attributes. Low-level details like edges and textures as well as higher-level abstractions like object shapes or categories can be included in the features. Convolutional layers or deep learning-inspired architectures that mimic the activity of the neocortex's visual processing regions (V1, V2, V4, IT) can be used by NeocortexAPI.
- NeocortexAPI guarantees effective sparse encoding or dimensionality reduction, enabling more successful similarity comparisons even though the feature vectors may be highly dimensional.

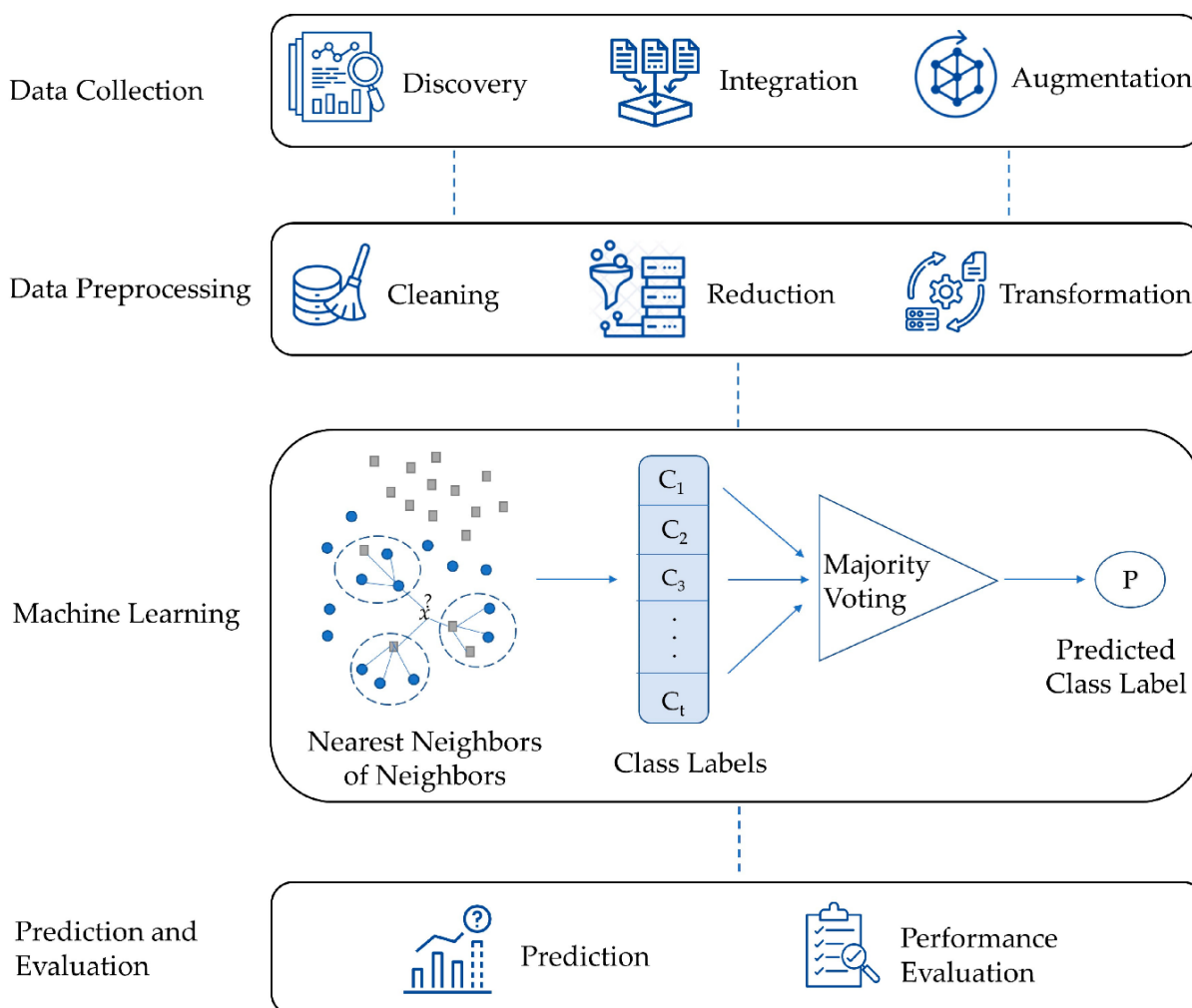


Fig6: High-Level K-Nearest Neighbors

3.3.2 Distance Metrics for Similarity Measurement

- The distance metric establishes how similarity is quantified in the KNN algorithm. In KNN, some frequently used distance metrics are: The Euclidean Distance calculates the distance in a straight line between two feature vectors. The neural-style embeddings of two images can be compared using Euclidean distance in NeocortexAPI.
- Cosine Similarity: This metric calculates the cosine of the angle formed by two vectors, emphasizing orientation over magnitude. When comparing high-level features in a neural embedding space produced by NeocortexAPI, this is especially helpful.
- Manhattan Distance: This method, which works well with images that have a grid-like structure, adds up the absolute differences between the feature vectors. KNN can determine which images have the most similar neocortical-style

representations by calculating the distances between the feature vectors that NeocortexAPI extracted.

3.3.3 KNN Search: Identifying Nearest Neighbors

- **Query image:** To start, a query image is sent through the feature extraction layers of the NeocortexAPI. As a result, a feature vector is created, which is used as a guide to locate related images.
- **Dataset comparison:** The KNN classifier contrasts the feature vectors of the dataset's images with those of the query image. The distance between feature vectors is calculated for every image in the dataset.
- **K-nearest neighbors:** The KNN algorithm chooses the K images with the smallest distances (i.e., the most similar images) after the distances have been calculated. These pictures are regarded as the query image's "nearest neighbors" according to the distance metric that was selected.
- **Image similarity ranking:** In the feature space, the images are arranged according to how close they are to the query image. The degree of similarity between the images increases with decreasing distance.

3.3.4 Handling Large Datasets with KNN in NeocortexAPI

- **Approximate Nearest Neighbor (ANN) search:** Exact KNN search can be computationally costly for large datasets. By approximating the nearest neighbors, ANN techniques like KD-Trees, Ball Trees, or Locality-Sensitive Hashing (LSH) can increase efficiency while lowering the computational load.
- **Dimensionality reduction:** The most important features for similarity comparisons can be preserved while the dimensionality of the feature space is decreased using methods like Principal Component Analysis (PCA) or Autoencoders in NeocortexAPI. This guarantees that even for complex, high-dimensional data, KNN search will continue to be effective.

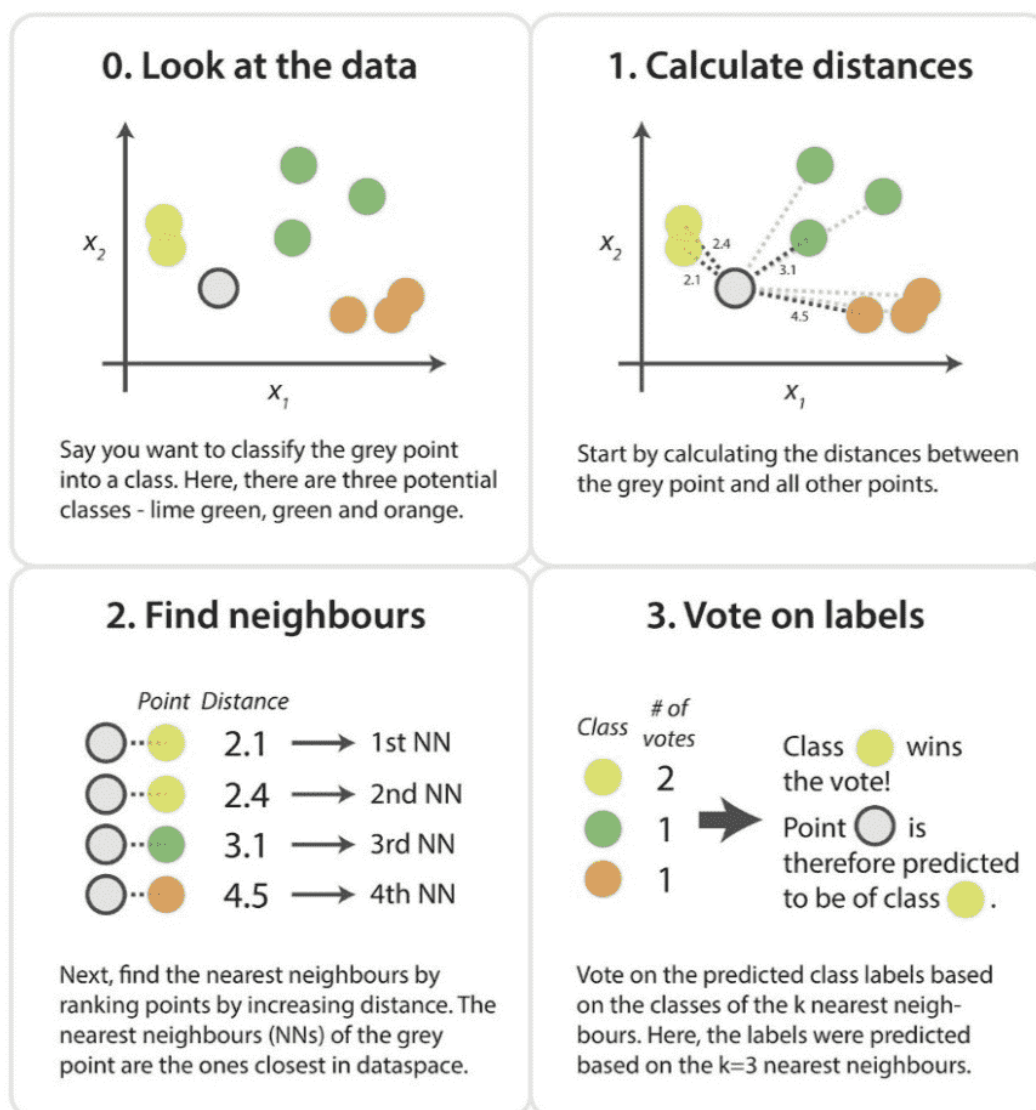


Fig7: Understanding K-Nearest Neighbors

3.4 Neocortical Mechanisms in KNN-Based Image Similarity

Important information about how the brain evaluates visual inputs and establishes similarity can be gained from the neocortical neural mechanisms at play. NeocortexAPI's method for image similarity directly applies these ideas:

3.4.1 Hierarchies of Features

- An analogy from V1 to IT cortex: To extract low-level to high-level features from images, NeocortexAPI uses a hierarchy of convolutional layers, much like the visual

cortex does when processing raw visual stimuli into progressively complex representations. These hierarchical features are subjected to KNN in order to compare images.

- Invariance to transformations: Despite variations in illumination, scale, and orientation, the neocortex can still identify objects. In a similar vein, NeocortexAPI encodes images so that the KNN classifier can recognize similar images even when they are displayed in different formats (such as scaled or rotated).

3.4.2 Sparse Coding and Cortical Columns

The theory that only a tiny portion of neurons fire in response to a specific stimulus is known as "sparse coding" in the neocortex. By concentrating on the most informative features and minimizing the number of features used to represent an image, NeocortexAPI imitates this process. This sparsity improves KNN's capacity to identify significant image similarities without being overtaken by extraneous information. Certain areas of an image, like lines or corners, are processed by cortical columns. By extracting local features from a localized feature space, NeocortexAPI replicates this. KNN determines similarity between images by comparing these localized features.

3.4.3 Pattern Completion and Separation

The brain uses pattern separation to discern between extremely similar but distinct inputs and pattern completion to identify known patterns from incomplete or noisy inputs in the neocortex. Similar ideas are applied by NeocortexAPI:

- Pattern completion: NeocortexAPI's KNN classifier uses high-level representations that capture the essence of the image to find the closest match when comparing a query image to a dataset, even if portions of the image are obscured or missing.
- Pattern separation: By concentrating on distinct features encoded in the high-level layers of NeocortexAPI's feature extraction procedure, KNN is able to distinguish between similar images.

3.5 Applications of KNN-Based Image Similarity in NeocortexAPI

There are numerous uses for KNN for image similarity in NeocortexAPI in a variety of fields:

3.5.1 Content-Based Image Retrieval (CBIR)

Users submit a query image to CBIR systems, which then use a database to find the most similar images. KNN in conjunction with NeocortexAPI enables reliable and effective retrieval based on high-level semantic features as opposed to pixel-by-pixel comparisons. This is helpful when looking through sizable image collections, such as art collections or databases for medical imaging.

3.5.2 Face Recognition and Object Detection

KNN can be used by NeocortexAPI to identify faces or objects in a dataset by utilizing deep feature representations that are modelled after neocortical processing. By comparing their abstract feature representations, the classifier determines which faces or objects are the most similar.

3.5.3 Systems for Visual Recommendations

Based on a query image, KNN and NeocortexAPI can recommend visually similar items to a user in recommendation systems (such as e-commerce platforms). By contrasting the visual characteristics of the query image with those of other database items, the system is able to identify comparable products.

Although NeocortexAPI's KNN-based image similarity is an effective method, there are drawbacks:

- **Curse of dimensionality:** Traditional KNN is hampered by the curse of dimensionality in high-dimensional spaces, where distances lose their significance. By concentrating on the most pertinent features, NeocortexAPI's dimensionality reduction and sparse coding help to lessen this problem.
- **Computational efficiency:** For large datasets, exact KNN search is computationally costly. NeocortexAPI can use approximate techniques like ANN to solve this and expedite the search.
- **Quality of feature representation:** The quality of the feature vectors that NeocortexAPI extracts has a significant impact on KNN performance. Accurate image similarity comparisons depend on the feature extraction layers being properly trained and adjusted.

An effective and biologically inspired technique for comparing images based on their high-level feature representations is offered by the combination of KNN and NeocortexAPI.

NeocortexAPI enables KNN to identify similar images in a semantically meaningful manner, even when those images are different in appearance or have undergone changes, by utilizing the hierarchical, sparse, and distributed processing mechanisms of the neocortex.

Numerous applications, such as recommendation systems, face recognition, and image retrieval, benefit from this method.

3.4 Steps in KNN Classification:

3.4.1 Selecting the number of neighbors (K):

- You choose a value for K that establishes the number of nearest neighbors that will be considered when classifying a new instance.
- A low K value (K=1, for example) may cause the classifier to overfit (be more sensitive to noise), whereas a high K value may cause underfitting (consider too many neighbors, some of which may be irrelevant points).

3.4.2 Distance Calculation:

The algorithm determines the distance between each point in the training set and a new instance in order to classify it.

When assessing how close neighbors are to one another, the distance metric selection is essential:

- The most widely used is Euclidean Distance, but improperly scaled features can produce misleading results, which is why feature scaling is so important.
- Other distance measures, like Manhattan Distance or Cosine Similarity, may be more suitable in some circumstances. Typical distance measurements consist of:

Euclidean Distance (most frequently used in KNN):

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Two points in the feature space.

$$d(p, q) = \sum_{i=1}^n |p_i - q_i|$$

$d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$
 where two points in the feature space are p and q .

Manhattan Distance (L1 norm):

$$d(p,q) = \sum_{i=1}^n |p_i - q_i|$$

Minkowski Distance:

Euclidean and Manhattan distances are generalized.

3.4.3 Identifying the K-nearest neighbors:

The algorithm chooses the top K nearest neighbors after sorting the training points by distance after the distances have been calculated.

- The bias-variance trade-off must be balanced, and this depends on the value of K.
- Low K (K = 1 or 3): Produces high variance and low bias. Although the model closely matches the training data, noise (overfitting) can affect it.
- A large K (such as K = 10 or 20) results in low variance and high bias. More general predictions will result from the model's smoothing over the data points (underfitting).
- In order to determine the ideal value, K is frequently selected through cross-validation.

3.4.4 Feature Scaling:

The algorithm uses a majority vote among the K nearest neighbors to determine the class label. The new instance is categorized under the most prevalent class among those nearby.

Scaling features is crucial because KNN depends on distance computations, particularly when the features have disparate units or ranges. One feature may dominate the distance calculation and skew the results if its range is significantly greater than another's. All features contribute equally to the distance calculation thanks to scaling.

3.5 KNN Variants and Improvements for Image Classification

To improve the standard KNN algorithm's performance in image classification tasks, a number of improvements have been suggested:

- **Hybrid Models:** To enhance performance, hybrid models that incorporate KNN with additional classifiers or dimensionality reduction strategies have been researched. For example, El-Yamany et al. (2012) enhanced the accuracy and robustness of standard KNN for image classification by combining it with support vector machines (SVMs).
- **Weighted KNN:** A weighted version of KNN, in which the classification decision is more heavily influenced by closer neighbors, was proposed by Sheikhpour et al. (2017). In image data, where local variations may affect the standard KNN's performance, this is especially helpful.
- **Fast KNN with Approximate Methods:** Datar et al. (2004) introduced approximate KNN algorithms such as Locality-Sensitive Hashing (LSH) to overcome the computational bottlenecks of the KNN in large-scale image classification task. These techniques are frequently applied in big image datasets such as ImageNet and effectively approximate the nearest neighbors.
- **Fuzzy KNN:** Developed by Keller et al. (1985), fuzzy KNN uses fuzzy membership values instead of a strict majority vote to determine classification. This method works particularly well for image classification tasks where it is difficult to distinguish between different classes.
- **Comparing Algorithm to Others:** Mixed results are found when KNN is compared to other classifiers such as SVM, decision trees and neural networks. KNN struggles with noise and irrelevant features, but it frequently performs better in situations with small to medium-sized datasets and clearly defined boundaries.
- **Scalability and Efficiency:** Although KNN is typically computationally costly because it must compute distances for every instance, performance problems are lessened by the Neocortex API's optimizations (such as approximate nearest neighbors), which enable it to process larger datasets more efficiently and successfully.

The K-Nearest Neighbors (KNN) algorithm is one of the simplest and most approachable machine learning algorithms for classification as well as regression problems. It belongs to the instance-based learning or lazy learning algorithm family since it uses the training set's closest data points to make predictions rather than learning a model explicitly.

An effective method for identifying and categorizing patterns in time-series data is offered by the combination of KNN and HTM. KNN's capacity to categorize those predictions using historical examples improves HTM's capacity to forecast temporal sequences. This hybrid method works well for tasks like anomaly detection, time-series forecasting, and pattern

recognition where accurate predictions depend on both spatial and temporal dependencies. When incorporated into the Neocortex API, the KNN classifier offers a potent tool for a range of uses, especially in pattern recognition and anomaly detection. The difficulties with dimensionality and computational efficiency counterbalance its advantages in efficiency and simplicity. Continuous improvements to the algorithm and its integration into the Neocortex framework are anticipated to increase its performance and applicability in challenging real-world situations as research advances.

Future research could look into integrating additional machine learning classifiers into a HTM framework or optimizing KNN for large datasets (for example, by using approximate nearest neighbors).

Over the years, KNN's use in image classification has changed from simple pixel-based approaches to more complex feature extraction and dimensionality reduction strategies. Despite its computational demands and difficulties with high-dimensional data, KNN has remained relevant due to its ease of use, adaptability, and efficiency in smaller-scale tasks. The usefulness of KNN has been increased by contemporary improvements like weighted KNN, approximate KNN algorithms, and hybrid models, especially when dealing with big datasets and image classification problems. However, KNN is now used as a benchmark or supplementary technique in image classification as deep learning techniques have taken over.

A temporal prediction based on learned sequences is represented by the predicted SDR produced by HTM's Temporal Memory. We can further categorize these anticipated SDRs into established groups by employing a K-Nearest Neighbors (KNN) classifier. In this combination, KNN offers a straightforward and efficient method of classifying related patterns based on historical data, while HTM can take advantage of its capacity to model temporal dependencies.

In order to increase efficiency, future research may concentrate on improving KNN using sophisticated methods like dimensionality reduction and feature selection. KNN as well as other machine learning methods are combined in hybrid models to increase accuracy and robustness. Neocortex API's real-time processing features allow for instant insights from streaming data.

By combining these two techniques, the workflow described in this documentation provides a potent method for anomaly detection, sequence predictions and classification in the time-series data, and other temporal pattern recognition activities.

KNN Applications in Image Data Today:

Even though more sophisticated deep learning techniques frequently outperform KNN in contemporary image classification tasks, in some applications it can still be used as a baseline or in conjunction with other methods:

- **Recognition of Handwritten Characters:** KNN has long been used for tasks such as optical character recognition (OCR), especially for handwritten characters and digits, where it yields dependable results with little preprocessing.
- **Fine-Grained Image Classification:** KNN is frequently utilized in hybrid systems with deep learning features or as a component of more complex models for fine-grained image classification tasks (such as identifying bird species).
- **Medical Imaging:** KNN is still utilized in medical imaging tasks like disease detection and tumour classification. To achieve competitive results, Suzuki et al. (2006) used KNN to classify images of breast cancer using extracted texture features.

3.6 Advantages and Applications:

- **Intuitive and Simple:** Simple to comprehend and apply.
- **Non-parametric:** KNN doesn't assume anything about the distribution of the underlying data.
- **Versatile:** Suitable for both regression and classification.
- **Recommendation systems:** These tools can be used to identify a user's or item's closest neighbors in order to provide tailored suggestions.
- **Handwriting detection:** Using pixel data, KNN can be used to categorize handwritten characters.
- **Medical diagnosis:** Based on the dataset's proximity to other cases that are similar, this method is used in domains such as cancer detection.

3.7 Challenges and Considerations:

- **High Computational Cost:** Due to KNN's lazy learning style, all training images must be calculated at prediction time, and the entire dataset must be stored in memory. Because of this, KNN is computationally costly, especially when dealing with big image datasets. In order to increase the computational efficiency of KNN in high-dimensional spaces, Indyk and Motwani (1998) introduced to approximate nearest neighbor search algorithms, like KD-trees and ball trees. For big datasets, KNN can be sluggish, particularly during prediction.
- **Noise Sensitivity:** KNN is susceptible to noisy data, particularly in the image domain where minor misalignments, lighting changes, or occlusion can result in subpar classification. By allocating distinct weights to neighbors according to their distance from the query point, distance-weighted KNN was developed to counteract this and give closer neighbors greater control over the classification. Noisy or irrelevant features have the

potential to skew the distance calculation because all features contribute to the distance metric.

- **Affected by feature scaling:** For consistent outcomes, features must be normalized or standardized.
- **Distance Metric:** For binary vectors such as SDRs, hamming distance is frequently employed. However, other distance metrics (like cosine similarity) may be suitable based on as well as the problem domain type of SDRs.
- **Large Dimensionality of SDRs:** SDRs are usually very high-dimensional (e.g., 2048 or more bits) in real-world applications. In order to scale KNN to large datasets, effective search techniques (such as approximate nearest neighbors) may be required because computing distances between high-dimensional vectors can be computationally costly.
- **Class Imbalance:** KNN may favour the more frequent classes if the labels for historical SDRs are unbalanced, meaning that some classes have significantly more instances than others. This problem can be lessened by employing strategies like weighted KNN, under sampling, and oversampling.
- **KNN Hyperparameters** One crucial hyperparameter that needs to be adjusted is the number of neighbors (K). While a large K could result in over-smoothing, a small K could produce noisy predictions. The best K for the task can be found with the aid of cross-validation.
- **Curse of Dimensionality:** When using raw pixel values, image data frequently contains a large number of dimensions (features). The performance of KNN is impacted by the distances between data points becoming less significant as dimensionality rises. Liu et al. (2013) addressed this by applying KNN after first utilizing dimensionality reduction techniques such as PCA and Linear Discriminant Analysis (LDA), which greatly enhanced classification performance.

3.7.1 Complexity of Time:

- **Training complexity:** Since KNN doesn't actually perform any training, the time complexity for the training phase is $O(1)O(1)O(1)$.
- **Prediction complexity:** Because KNN must calculate the distance between every point in the training sets and that to the query point while making predictions, its time complexity is $O(n \times d)O(n \times d)O(n \times d)$, where 'ddd' is the number of features and 'nnn' are the number of training points. For large datasets, this may be slow.

3.7.2 Memory Utilization:

KNN is an instance-based learner, it needs to store the complete training dataset, which may be a drawback for applications that use a lot of data. Because KNN requires to store the entire dataset during inference, it can consume a lot of memory, particularly when working with large datasets.

3.7.3 Dimensionality's Curse:

KNN may perform poorly in high-dimensional spaces due to its sensitivity to unimportant features. As so will the number of features (or dimensions) increases, so will the distance between any two points will increase, making all points appear much farther apart. This phenomenon makes it harder for KNN to distinguish between relevant and irrelevant neighbors in high-dimensional spaces.

3.7.4 Handling Unbalanced Information:

When there is imbalanced class, KNN may be heavily in favour of the majority classes because it relies on counting neighbors. Techniques like distance-weighted KNN, which give more weight to neighbors closer to the query point, can help reduce this.

4. Methodology

4.1 Research Design

The Neocortex API is used to design and integrate the KNN (K-Nearest-Neighbor) Classifier. To train the model, it receives a series of values and preassigned labels. The user can provide unclassified sequences that require labelling after the model (a dictionary mapping of labels to their sequences) has been trained. The following flowchart will help us understand the flow of the drafted experiment.

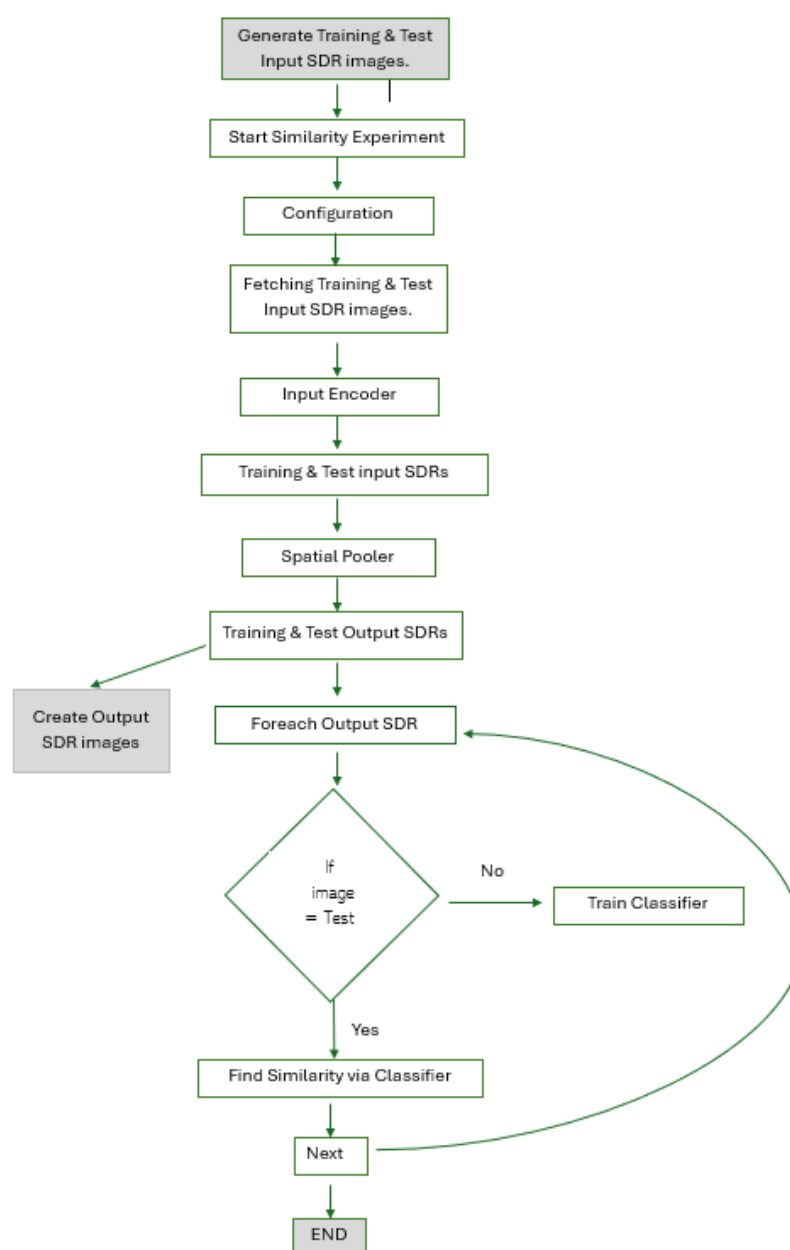


Fig8: Flowchart of experiment

4.2 Data Collection Methods

4.2.1 Generate Training & Test Input SDR images

In the experiment we are using the images with 30 x 30 pixels, which might be difficult to produce with exact dimensions and to have a desired input.

So, Input SDR is defined in 2D array and then created to images, so we have visual representation of the inputs.

- The below method is used to create input sdrs. It creates input images from 2D array of 1 and 0 values. The "inputSdrDirectoryPath" is the path to store input images.

```
public async Task CreateInputSdrs(string inputSdrDirectoryPath)
```

- In the file location if the Inputsdrs folder (this folder stores old input images which are converted from 2D array) exists then it deletes the folder and creates the new 'inputsdrs' folder (this folder stores current input images which are converted from 2D array).

```
if(Directory.Exists(inputSdrDirectoryPath))
{
    Directory.Delete(inputSdrDirectoryPath, true);
}
Directory.CreateDirectory(inputSdrDirectoryPath);
```

- It is calling each input sdr which is defined in InputSdrData.cs program. Get input SDR data in 2D arrays of 1 and 0 values

```
var inputSdrs = InputSdrData.GetInputSdrs();
```

- Input is defined in 2D array, instead of giving images we use this method, so we have the exact dimension of image and find similarity have more accuracy.

Key and Value with Key being name of the image and value being the 2D array of 1's and 0's

4.3 Data Analysis Method

4.3.1 Start Similarity Experiment

- First, it fetches and reads input images from the input SDR folder. Then, it generates output SDRs using input data. Then, we save the output SDRs as images. (For visualization, not necessary for the experiment). Then, it trains KNN classifier using training output SDRs and predicts test output SDRs. Finally, we display the predicted output SDRs, with its similarity compared to test output SDRs.

```
public async Task RunExperiment(string inputSdrsFolderPath, BinarizerParams
imageEncoderSettings)
{
    _logger.LogInformation($"Hello NeocortexApi! Running
{nameof(SimilarityExperiment)}");
```

- Fetch input SDRs file names from the input SDRs folder.

```
int inputBits = imageEncoderSettings.ImageHeight *
imageEncoderSettings.ImageWidth;

int numColumns = inputBits;

var inputSdrs = GetInputSdrs(inputSdrsFolderPath);
```

4.3.2 Configuration

Currently, in HTMConfig we used the standard configuration as mentioned below, however we see in later section how changes in Potential Radius and Activation Threshold affects the similarity percentage.

```
CellsPerColumn = 30,

GlobalInhibition = true,

LocalAreaDensity = -1,

NumActiveColumnsPerInhArea = 0.02 * numColumns,

PotentialRadius = (int)(0.15 * inputBits),

//InhibitionRadius = 15,

MaxBoost = 10.0,
```

```

DutyCyclePeriod = 25,
MinPctOverlapDutyCycles = 0.75,
MaxSynapsesPerSegment = (int)(0.02 * numColumns),

ActivationThreshold = 15,
SynPermConnected = 0.5,

// Learning is slower than forgetting in this case.
PermanenceDecrement = 0.25,
PermanenceIncrement = 0.15,

// Used by punishing of segments.
PredictedSegmentDecrement = 0.1

```

In Homeostatic Plasticity Controller configuration, we have set the minimum cycles as 3 times the count of input sdrs and maximum cycles as 4.5 times the count of input sdrs

```

var numUniqueInputs = inputSdrs.Count;

var homeostaticPlasticityControllerConfiguration = new
HomeostaticPlasticityControllerConfiguration()
{
    MinCycles = numUniqueInputs * 3,
    MaxCycles = (int)((numUniqueInputs * 3) * 1.5),
    NumOfCyclesToWaitOnChange = 50
};

```

4.3.3 Fetching Training & Test Input SDR images

Fetch input image file names from the input SDRs folder.

```
var inputSdrFilePaths = GetInputSdrFilePaths(inputSdrsFolderPath);
```

Fetch all the file paths from the input SDR folder. Get the file name without extension.

4.3.4 Input Encoder

Initialize image encoder. It takes the image settings to read input images and create one dimensional array of input.

```
var encoder = new ImageEncoder(imageEncoderSettings);
```

4.3.5 Training & Test input SDRs

Initialize output SDRs dictionary to store output SDRs with key as input SDR file name. Generate output SDRs by passing each input image file paths to image encoder and spatial pooler. Decode input image to 1D array of 1 and 0 values. Send that input to spatial pooler to generate output SDR. Add output SDR to the dictionary with key as input image file name.

4.3.6 Spatial Pooler

Create output SDR images from output SDRs which is a list of active column indices. First, it recreates a folder to store output SDR images.

Then, it creates output SDR images by setting the pixel value to 255 (white) if the active column index is present in the output SDR.

Set all other pixel values to 0 (black). Then, the output SDR images are saved in the output SDR folder.

```
Initialized spatial poller
Output SDR generated [337,342,384,387,399,400,403,455,461,466,474,497,511,514,533,534,541,548] for test_backslash_signs
Output SDR generated [387,388,396,398,399,400,403,406,450,454,476,491,510,514,534,538,541,548] for test_chevron_down_signs
Output SDR generated [342,369,386,387,403,413,455,461,466,467,469,511,514,529,533,534,540,548] for test_chevron_up_signs
Output SDR generated [362,366,388,421,435,444,450,456,458,464,470,474,498,502,505,515,520,558] for test_number_eight
Output SDR generated [366,378,397,412,435,444,458,462,464,467,470,474,498,502,514,517,520,558] for test_number_four
```

Fig12: Generating Output SDR via Spatial pooler

5. Results and Discussion

5.1 Generate Output SDRs

- Generate output SDRs by passing input SDR image file paths to image encoder and spatial pooler.
- The image encoder reads the image and converts it to a binary array. The spatial pooler computes the active columns for the input SDR. The output SDR is the list of active column indices.

```
logger.LogInformation("Generating Output SDRs.");

var outputSdrs = GenerateOutputSdrs(
    htmConfig,
    homeostaticPlasticityControllerConfiguration,
    encoder,
    inputSdrs);
```

5.2 Creating Output SDRs Images

- Save output SDR images. For visualization, not necessary for the experiment.
- Create output SDR images, by passing the output SDRs and image settings.

```
_logger.LogInformation("Creating Output SDR Images.");

var outputSdrFolderPath = "./OutputSdrs";

await CreateOutputSdrImages(
    outputSdrFolderPath,
    outputSdrs,
    imageEncoderSettings.ImageHeight,
    imageEncoderSettings.ImageWidth);
```

```

Creating Output SDR Images.
Output SDR image created at ./OutputSdrs\test_backslash_signs.png
Output SDR image created at ./OutputSdrs\test_chevron_down_signs.png
Output SDR image created at ./OutputSdrs\test_chevron_up_signs.png
Output SDR image created at ./OutputSdrs\test_number_eight.png
Output SDR image created at ./OutputSdrs\test_number_four.png

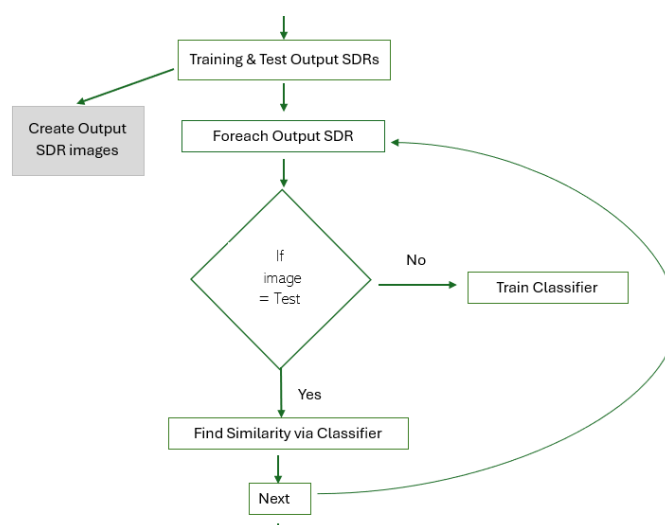
```



Fig13: Created Output SDR images

5.3 Training & Test Output SDRs

Train KNN classifier using training output SDRs and Predict test output SDRs



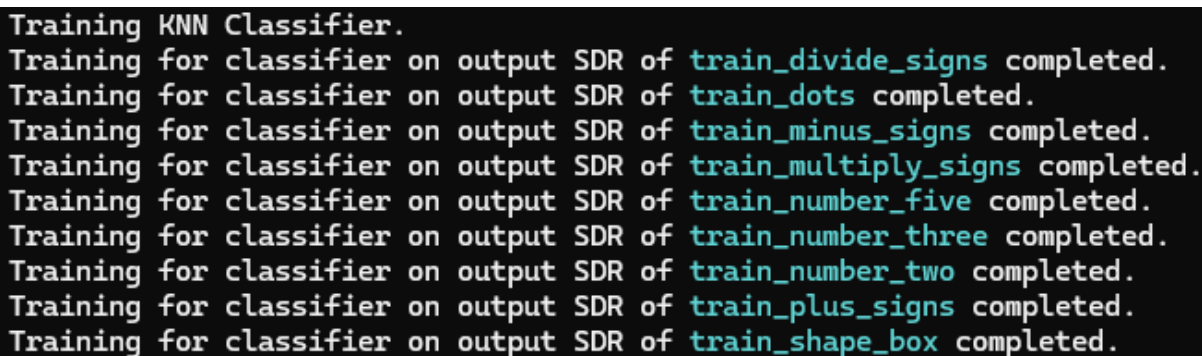
We are assigning KNeighborsClassifier, here we will call all the training output sdr from the output sdrs folder.

Initialize KNN classifier to train and predict output SDRs. Train KNN classifier using training output SDRs

```
var classifier = new KNeighborsClassifier<string, int[]>();
foreach (var trainingOutputSdr in outputSdrs.Where(x => x.Key.Contains("train")))
```

The classifier will then learn and get trained be output SDRs.

```
classifier.Learn(trainingOutputSdr.Key, trainingOutputSdr.Value.Select(x => new
Cell(0, x)).ToArray());
```



```
Training KNN Classifier.
Training for classifier on output SDR of train_divide_signs completed.
Training for classifier on output SDR of train_dots completed.
Training for classifier on output SDR of train_minus_signs completed.
Training for classifier on output SDR of train_multiply_signs completed.
Training for classifier on output SDR of train_number_five completed.
Training for classifier on output SDR of train_number_three completed.
Training for classifier on output SDR of train_number_two completed.
Training for classifier on output SDR of train_plus_signs completed.
Training for classifier on output SDR of train_shape_box completed.
```

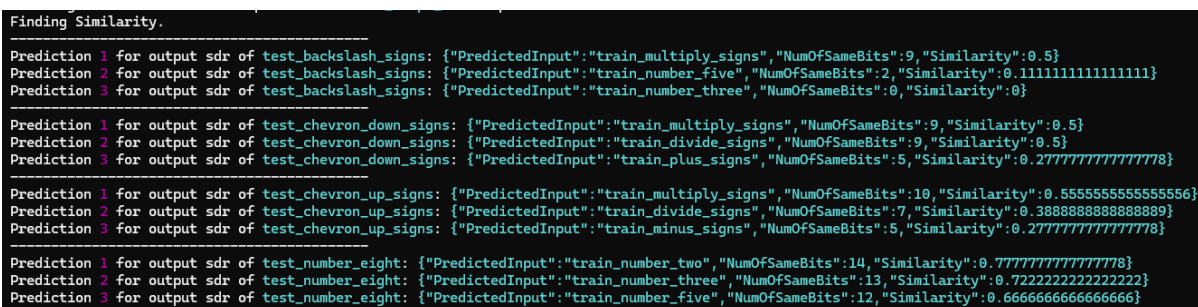
Fig14: Training Output SDRs

5.4 Find Similarity via Classifier

We will call all the test output sdrs from the output sdrs folder.

```
foreach (var testOutputSdr in outputSdrs.Where(x => x.Key.Contains("test")))
```

After the prediction is complete it will take top 3 prediction based on highest similarities, with the number of similar bits and the percentage of similarity.



```
Finding Similarity.
Prediction 1 for output sdr of test_backslash_signs: {"PredictedInput":"train_multiply_signs", "NumOfSameBits":9, "Similarity":0.5}
Prediction 2 for output sdr of test_backslash_signs: {"PredictedInput":"train_number_five", "NumOfSameBits":2, "Similarity":0.1111111111111111}
Prediction 3 for output sdr of test_backslash_signs: {"PredictedInput":"train_number_three", "NumOfSameBits":0, "Similarity":0}

Prediction 1 for output sdr of test_chevron_down_signs: {"PredictedInput":"train_multiply_signs", "NumOfSameBits":9, "Similarity":0.5}
Prediction 2 for output sdr of test_chevron_down_signs: {"PredictedInput":"train_divide_signs", "NumOfSameBits":9, "Similarity":0.5}
Prediction 3 for output sdr of test_chevron_down_signs: {"PredictedInput":"train_plus_signs", "NumOfSameBits":5, "Similarity":0.2777777777777778}

Prediction 1 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_multiply_signs", "NumOfSameBits":10, "Similarity":0.5555555555555556}
Prediction 2 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_divide_signs", "NumOfSameBits":7, "Similarity":0.3888888888888889}
Prediction 3 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_minus_signs", "NumOfSameBits":5, "Similarity":0.2777777777777778}

Prediction 1 for output sdr of test_number_eight: {"PredictedInput":"train_number_two", "NumOfSameBits":14, "Similarity":0.7777777777777778}
Prediction 2 for output sdr of test_number_eight: {"PredictedInput":"train_number_three", "NumOfSameBits":13, "Similarity":0.7222222222222222}
Prediction 3 for output sdr of test_number_eight: {"PredictedInput":"train_number_five", "NumOfSameBits":12, "Similarity":0.6666666666666666}
```

Fig15: Output Similarity prediction

The Neocortex API has been used to integrate the KNN (K-Nearest-Neighbor) Classifier in HTM. The model was trained, by receiving a series of values/dataset and preassigned labels. In the experiment I have provided different inputs such as divide, multiply, minus signs and

different numbers for training the classifier and backlash sign, chevron up/down and some integers for the testing of the classifier, the classifier has converted the input datasets into input SDR images, later these input SDR images were used and created an output SDR using Spatial pooler.

These images were later used to find the similarities and accuracy between training SDRs and test SDRs. The system has predicted the similarity, and it let us know the similarity percentages and similar bits of the top three predictions for each test SDRs.

By this we were able to understand the prediction of Similar patterns of data with KNN classifier in HTM (Hierarchical Temporal Memory).

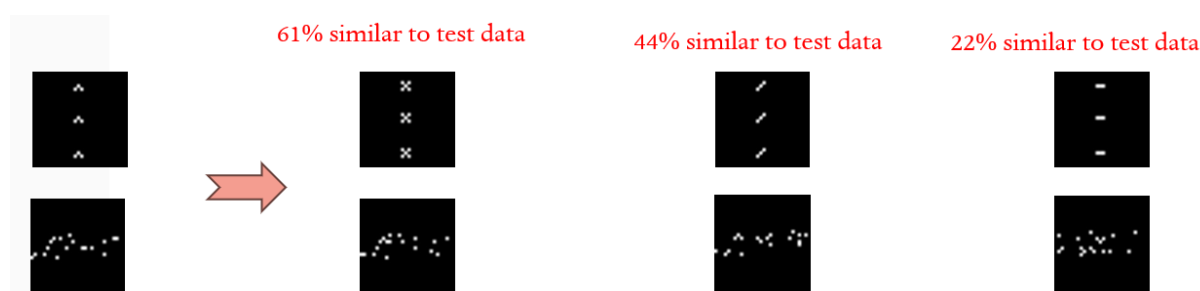
This approach makes use of KNN's ease of classification and HTM's resilience in temporal anomaly detection. This framework has been used to examine the prediction accuracy of similar patterns in datasets that contain repetitive patterns. Overtime the KNN classifier improved the pattern similarity prediction, via increasing efficiency and accuracy when combined with HTM's temporal learning.

5.5 Analyzing the Similarity

Train KNN classifier using training output SDRs and Predict test Output SDRs for top 3 values

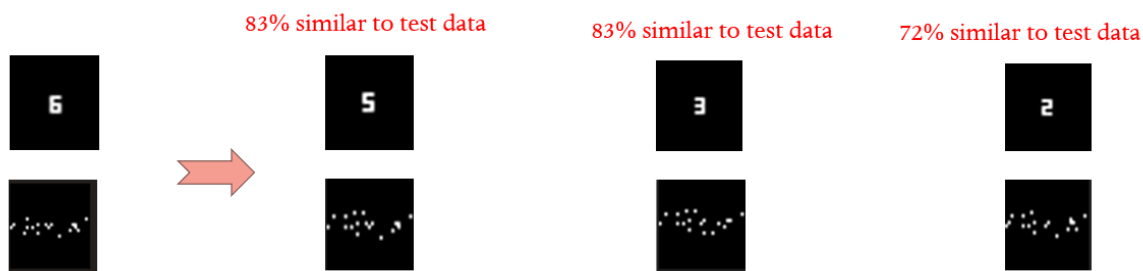
```
[12:27:24 INF] -----
[12:27:24 INF] Prediction 1 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_multiply_signs","NumOfSameBits":11,"Similarity":0.6111111111111112}
[12:27:24 INF] Prediction 2 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_divide_signs","NumOfSameBits":8,"Similarity":0.4444444444444444}
[12:27:24 INF] Prediction 3 for output sdr of test_chevron_up_signs: {"PredictedInput":"train_minus_signs","NumOfSameBits":4,"Similarity":0.2222222222222222}
[12:27:24 INF] -----
```

If we take chevron_up_signs: this image is similar to with Multiply_sign, divide_sign and then minus_sign



If we take another example; Number_six : this image is similar to with Number_five, Number_two and then Number_three.

```
[05:34:25 INF] -----
[05:34:25 INF] Prediction 1 for output sdr of test_number_six: {"PredictedInput":"train_number_five","NumOfSameBits":15,"Similarity":0.8333333333333334}
[05:34:25 INF] Prediction 2 for output sdr of test_number_six: {"PredictedInput":"train_number_two","NumOfSameBits":15,"Similarity":0.8333333333333334}
[05:34:25 INF] Prediction 3 for output sdr of test_number_six: {"PredictedInput":"train_number_three","NumOfSameBits":13,"Similarity":0.7222222222222222}
[05:34:25 INF] -----
```



The Classifier is confirming that similar input patterns are similarly encoded by Spatial Pooler.

This is done by predicting the similar output SDRs which are created by input patterns that are similar.

5.6 Adjusting HTM Configuration:

Now, we have reduced the potential radius and increased the activation threshold.

By reducing the potential radius, each column focuses on a specific, smaller region of the input. This localized focus makes the columns more specialized, increasing the likelihood that similar input patterns in the same area activate similar columns. This specialization allows HTM to better distinguish small, nuanced differences in specific regions of the input space.

Increasing the activation threshold in HTM's spatial pooler improves similarity prediction by enforcing stricter activation criteria. This selectivity ensures that similar inputs consistently activate the same columns, leading to stable and distinctive SDRs for similar patterns. A higher activation threshold strengthens the spatial pooler's ability to accurately predict and recognize similarities.

```

HtmConfig htmConfig = new(inputDims: new[] { inputBits }, columnDims: new[] { numColumns })
{
    Random = new ThreadSafeRandom(42),

    CellsPerColumn = 30,
    GlobalInhibition = true,
    LocalAreaDensity = -1,
    //LocalAreaDensity = -0.5,
    NumActiveColumnsPerInhArea = 0.02 * numColumns,
    //PotentialRadius = (int)(0.15 * inputBits),
    PotentialRadius = (int)(0.13 * inputBits),
    //InhibitionRadius = 15,

    MaxBoost = 10.0,
    DutyCyclePeriod = 25,
    MinPctOverlapDutyCycles = 0.75,
    MaxSynapsesPerSegment = (int)(0.02 * numColumns),

    //ActivationThreshold = 15,
    ActivationThreshold = 30,
    SynPermConnected = 0.5,

    // Learning is slower than forgetting in this case.
    PermanenceDecrement = 0.25,
    PermanenceIncrement = 0.15,

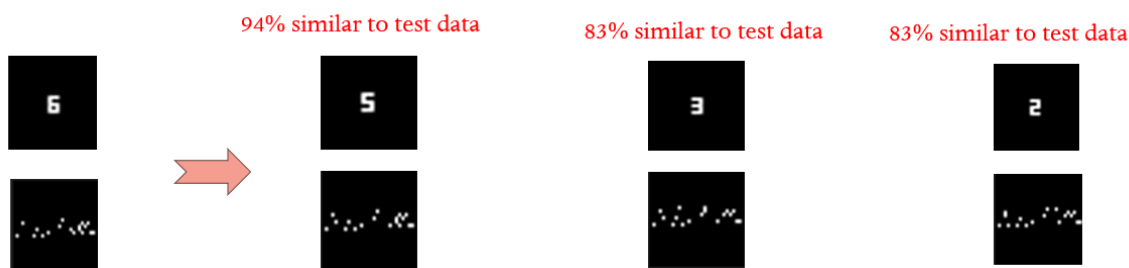
    // Used by punishing of segments.
    PredictedSegmentDecrement = 0.1
};

```

Fig16: Updated HTMConfig

This leads to the increase the similarity of Number six to Number five from 83% to 94%

```
[05:57:14 INF] Prediction 1 for output sdr of test_number_six: {"PredictedInput": "train_number_five", "NumOfSameBits": 17, "Similarity": 0.9444444444444444}
[05:57:14 INF] Prediction 2 for output sdr of test_number_six: {"PredictedInput": "train_number_three", "NumOfSameBits": 15, "Similarity": 0.8333333333333334}
[05:57:14 INF] Prediction 3 for output sdr of test_number_six: {"PredictedInput": "train_number_two", "NumOfSameBits": 15, "Similarity": 0.8333333333333334}
```



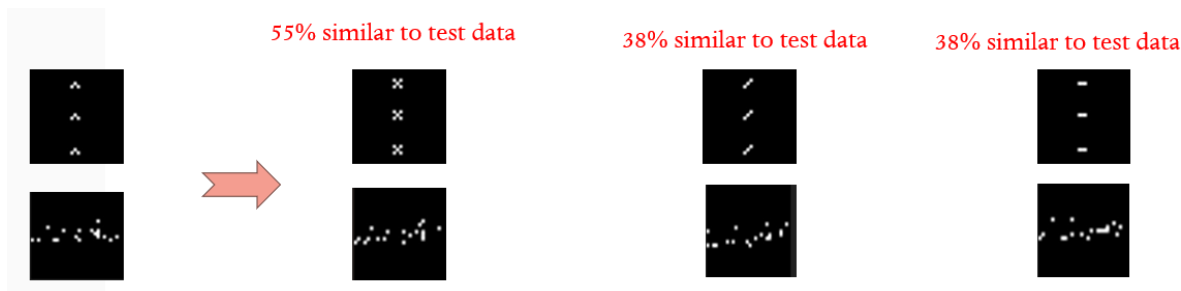
Similarity, it has also increased the similarities for number four image:

```
Prediction 1 for output sdr of test_number_four: {"PredictedInput": "train_number_five", "NumOfSameBits": 9, "Similarity": 0.5}
Prediction 2 for output sdr of test_number_four: {"PredictedInput": "train_number_three", "NumOfSameBits": 9, "Similarity": 0.5}
Prediction 3 for output sdr of test_number_four: {"PredictedInput": "train_number_two", "NumOfSameBits": 8, "Similarity": 0.4444444444444444}
```

```
Prediction 1 for output sdr of test_number_four: {"PredictedInput": "train_number_three", "NumOfSameBits": 12, "Similarity": 0.6666666666666666}
Prediction 2 for output sdr of test_number_four: {"PredictedInput": "train_number_five", "NumOfSameBits": 11, "Similarity": 0.6111111111111112}
Prediction 3 for output sdr of test_number_four: {"PredictedInput": "train_number_two", "NumOfSameBits": 10, "Similarity": 0.5555555555555556}
```

However, for chevron_up_signs: the similarity percentage has reduced

```
[05:57:14 INF] Prediction 1 for output sdr of test_chevron_up_signs: {"PredictedInput": "train_multiply_signs", "NumOfSameBits": 10, "Similarity": 0.5555555555555556}
[05:57:14 INF] Prediction 2 for output sdr of test_chevron_up_signs: {"PredictedInput": "train_divide_signs", "NumOfSameBits": 7, "Similarity": 0.3888888888888889}
[05:57:14 INF] Prediction 3 for output sdr of test_chevron_up_signs: {"PredictedInput": "train_minus_signs", "NumOfSameBits": 7, "Similarity": 0.3888888888888889}
```



Potential reasoning would be because the input bits are sparsely placed which might be the reason to reduced similarity percentage.

6. Conclusion

In this study, the Neocortex API was integrated with the K-Nearest-Neighbor (KNN) classifier within the Hierarchical Temporal Memory (HTM) framework to explore pattern similarity prediction through Spatial Pooler-generated sparse distributed representations (SDRs). By encoding inputs as SDR images, the classifier was trained with labeled sequences, allowing it to predict the similarity of unclassified patterns effectively. The experiment demonstrated KNN's ability to recognize similar patterns based on encoded SDR images, with high accuracy observed in the top three similarity predictions. Adjustments to HTM's configuration, such as reducing the potential radius and increasing the activation threshold, improved prediction accuracy by making SDRs more consistent and selective for similar inputs.

The results showed that these adjustments enhanced the system's ability to accurately differentiate and classify similar patterns, as evidenced by increased similarity percentages in certain cases, such as the numbers six and five. However, in patterns where input bits were sparsely distributed, such as with the chevron-up symbol, similarity percentages decreased slightly, potentially due to the reduced overlap in active regions. Overall, this experiment highlights how HTM's biologically inspired mechanisms, combined with KNN's robust classification, can be fine-tuned to optimize similarity prediction across repetitive datasets.

Bibliography

- <https://github.com/ddobric/neocortexapi/blob/master/source/Documentation/SpatialPooler.md>
- Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory*.
- T. Yeh, K. Tollmar, and T. Darrell. Searching the web with mobile images for location recognition. In *CVPR* (2), pages 76–81, 2004.
- Numenta. (n.d.). Neocortex API Documentation.
- Aha, D. W., Kibler, D., & Albert, M. K. (1991). Instance-based learning algorithms. *Machine Learning*.
- Zhang, J., & Wang, S. (2019). A study on the effectiveness of KNN algorithm in anomaly detection. *Journal of Data Science*.
- <https://www.numenta.com/resources/research-publications/papers/htm-spatial-pooler-neocortical-algorithm-for-online-sparse-distributed-coding/>
- Hawkins, Jeff and Blakeslee, Sandra. *On Intelligence*. USA, St. Martin's Griffin (2005)
- Fan, Jialue and Xu, Wei and Gong, Yihong. Convolutional Neural Networks: Human Traffic. *IEEE Transactions about Neural Networks*. USA, 1610–1623 (2010).
- <https://www.sciencedirect.com/science/article/abs/pii/S0959438814000105>
- https://www.researchgate.net/figure/Pattern-completion-a-set-of-neocortical-patterns-activated-by-a-particular-experience_fig1_260999562
- Jahan Balasubramaniam, Gokul Krishnaa.C.Ba, Fangming Zhu (2015). Enhancement of Classifiers in HTM-CLA Using Similarity Evaluation Methods
- Singh Jal al, Anand and Singh, Vrijendra: Visual Object Tracking: state of art. *An International Journal of Computing and Informatics*. Ljubljana, Slovenia. Vol 3,227–247 (2011)
- Hassner, Tal and Mayzels, Viki and Zelnik-Manor, Lihi. About SIFT and its scale. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Rhode Island, USA (2012)
- Lowe, David G. Object Recognition based on Local Scale Invariant Features. *Computer Vision: internationalconference*. IEEE, Canada, 1150–1157 (1999).DOI:10.1109/ICCV.1999.790410
- Solem, Jan Erik. *Python: basics on Computer Vision*. O'Reilly Medias, USA (2012)
- P. Y. Pawar and S. Gawande, “A comparative study on different types of approaches to text categorization,” *International Journal of Machine Learning and Computing*, vol. 2, no. 4, p. 423, 2012.
- P. Zezula, G. Amato, V. Dohnal, and M. Batko. *Similarity Search: The Metric Space Approach*, volume 32 of *Advances in Database Systems*. Springer-Verlag, 2006.
- Ozturk Kiyak, E.; Ghasemkhani, B.; Birant, D. High-Level K-Nearest Neighbors (HLKNN): A Supervised Machine Learning Model for Classification Analysis. *Electronics* **2023**, *12*, 3828. <https://doi.org/10.3390/electronics12183828>
- D. George, “How the brain might work: A hierarchical and temporal model for learning and recognition,” Ph.D. dissertation, Stanford University, 2008.

- T. Tuytelaars and K. Mikolajczyk. Local invariant feature detectors: a survey. *Found. Trends. Comput. Graph. Vis.*, 3(3):177–280, 2008.
- T. Yeh, K. Tollmar, and T. Darrell. Searching the web with mobile images for location recognition. In *CVPR* (2), pages 76–81, 2004.
- Y. Zheng, M. Z. 0003, Y. Song, H. Adam, U. Buddemeier, A. Bissacco, F. Brucher, T.-S. Chua, and H. Neven. Tour the world: Building a web-scale landmark recognition engine. In *CVPR*, pages 1085–1092. IEEE, 2009.
- <https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>
- L. L. Dhande and G. K. Patnaik, “Analyzing sentiment of movie review data using naive bayes neural classifier,” *International Journal of Emerging Trends & Technology in Computer Science (IJETTCS)*, vol. 3, no. 4, 2014.
- https://www.mathworks.com/discovery/machinelearning/_jcr_content/thumbnail.adapt.1200.medium.svg/1491334105233.svg
- H.M Zelelew, A.T Papagiannakis, "A volumetric thresholding algorithm for processing asphalt concrete x-ray CT images". *International journal of pavement engineering*, Sept. 2007.
- Min-Ling, Zhi-Hua Zhou, "A K-nearest neighbour based algorithm for multilabel classification." *IEEE International Conference of Granular Computing*, July 2005; pp: 718-72
- A. Pentland, R.W.Picard, S.Scaroff. *Photobook: Content-based manipulation for image databases*. *International Journal of Computer Vision* 18 (3), 233-254, 1996.

Declaration of Authorship

I, the undersigned, hereby declare that I am the independent author and composer of this master's thesis.

All ideas and content derived from outside sources, whether directly or indirectly, are indicated as such. This thesis has referred to all of the materials and sources that were used.

The thesis has not been published and has not been submitted to any other examining body.

Erlenbach am Main, 06/11/2024

Place, date



Signed: Jayashree Venkataramana Regoti